

SystemVerilog / UVM

1~8단계 시각 학습 노트

대화에서 학습한 내용 + 회사 PDF 보충

객체지향 · 프로세스 통신 · Randomization · Interface / UVM 구조 · Object/Component · Factory · Phase/Objection

2026-10-07 갱신 | 다음 시작점: 9단계 Sequence

MD와 같은 본문으로 구성했다. 자료 보충 항목은 이해도를 확인한 내용과 구분하며, 모든 UVM 코드 예제의 실행 상태는 미실행이다.

목차와 읽는 방법

자료 범위와 현재 진도	3
SystemVerilog/UVM 1단계 - 객체지향 문법 복습	4
SystemVerilog/UVM 2단계 - 데이터 구조와 Process 통신	15
SystemVerilog/UVM 3단계 - Randomization과 Constraint	27
SystemVerilog/UVM 4단계 - Interface와 Simulation Timing	39
SystemVerilog/UVM 5단계 - UVM 개요와 전체 구조	53
SystemVerilog/UVM 6단계 - uvm_object와 uvm_component	65
SystemVerilog/UVM 7단계 - Factory와 utility macro	76
SystemVerilog/UVM 8단계 - Phase와 objection	87

자료 범위와 현재 진도

각 단계는 개념 그림 → 학습 내용 → PDF 보충 → 혼동한 부분/확인 문제 순으로 읽는다. 긴 코드 박스의 "계속"은 페이지 분할이다. 원문은 MD에서 확인한다.

회사 자료: `_uvm_tb_240705_214257.pdf`. 본문에 적은 쪽수는 책의 인쇄 쪽수다. PDF 파일 페이지는 책 쪽수 + 11이며 해당 단원의 그림·코드를 학습 흐름에 맞춰 재구성했다.

단계	대화 학습 + 보충 초점	회사 PDF 연결
1	handle, 복사, 상속, \$cast	01.03.03 / 01.03.05
2	array/queue, fork, event, mailbox	01.02.04~06 / 01.03.01 / 01.03.06
3	randomize, constraint, 배열 조건	01.03.04 / 02.04
4	virtual interface, clocking, sampling	01.03.02 / 01.03.05
5	환경 구조와 데이터 흐름	02.01~04
6	상속, parent, hierarchy, phase	02.01~04 / 02.06
7	factory, field 자동화, copy/compare	02.04 / 02.06
8	phase, 완료 조건, objection	02.01 / 02.02 / 02.09

현재는 8단계의 기본 이론과 짧은 코드 해석까지 진행했다. 9단계는 도입 설명만 했고 확인 문제는 미응답이다. 다음 학습은 이론을 먼저 설명하고 확인 문제 하나씩 푸는 방식으로 이어간다.

API 이름과 phase 방향 등은 UVM 1.2 reference를 함께 확인했다. 회사 PDF에서 생략되거나 혼동하기 쉬운 설명은 각 보충 항목에서 바로잡았다. 공식 API 링크는 7·8단계 참고 자료에 있다.

SystemVerilog/UVM 1단계 - 객체지향 문법 복습

handle 대입

a = b

두 handle이 같은 바깥 object를 참조

shallow copy

b = new a

바깥 object는 새로 생성
내부 handle은 공유

deep copy

내부도 생성 + 값 복사

바깥과 내부 object를 각각 독립 구성

Handle은 참조다. Object의 값 복사와 handle 공유를 구분한다.

학습 범위: class/object/handle부터 this/super까지. 이 노트는 대화에서 실제로 다룬 내용을 정리한 것이며, 코드는 개념 설명용입니다. simulator에서 실행해 검증한 결과는 아닙니다.

1. class, object, handle, constructor

- **class**: property와 method를 정의하는 설계도.
- **object**: new()로 생성된 class의 실제 instance.
- **handle**: object를 가리키는 변수. handle 선언만으로 object가 생성되지는 않는다.
- **function new(...)**: object 생성 시 실행되는 constructor. 직접 정의하지 않아도 기본 new()로 object를 만들 수 있지만, 초기화 동작이 필요하다면 작성한다.
- null handle로 property나 method에 접근하면 안 된다.

SystemVerilog / UVM

```
class Packet;
  int data;
  function new(int value = 5);
    data = value;
  endfunction
endclass

Packet p;
// 여기서 p는 null: object는 아직 없다.
p = new(10);
```

Packet p;의 p는 object가 아니라 handle이다. p = new(10);에서 object가 하나 생성되고 p가 그 object를 가리킨다.

2. handle assignment와 object copy

SystemVerilog / UVM

```
Packet a, b;
a = new(10);
b = a;      // handle assignment: 새 object 없음
b.data = 20; // a.data도 20으로 관찰됨
a = new(30); // 두 번째 object 생성. b는 첫 object를 계속 가리킴
```

b = a는 값이 담긴 object의 복사가 아니라 handle 값의 대입이다. 여러 handle이 같은 object를 가리킬 수 있다. 반대로 a = null은 a의 연결만 끊으며, 같은 object를 가리키는 다른 handle에는 영향을 주지 않는다.

Shallow copy와 deep copy

```
SystemVerilog / UVM
class Header;
    int id;
endclass

class Frame;
    int data;
    Header hdr;
endclass

Frame x, y;
x = new();
x.hdr = new();
y = new x; // 바깥 Frame은 새 object, 내부 hdr handle은 공유
```

연산	바깥 Frame	내부 Header
y = x	같은 object 공유	같은 object 공유
y = new x	새 object 생성	handle만 복사되어 공유
deep copy	새 object 생성	new()로 별도 생성하고 값 복사

```
SystemVerilog / UVM
y = new();
y.data = x.data;
y.hdr = new();
y.hdr.id = x.hdr.id; // int 값 복사; Header handle은 변경되지 않음
```

`y.hdr.id = x.hdr.id`는 int 값 복사이고, `y.hdr = x.hdr`는 handle assignment이다. 이미 `y.hdr = new()`를 했어도 뒤에서 `y.hdr = x.hdr`를 하면 다시 내부 object를 공유한다.

3. inheritance, overriding, polymorphism

SystemVerilog / UVM

```
class Transaction;
    int id;
    virtual function void print();
        $display("base id=%0d", id);
    endfunction
endclass

class WriteTransaction extends Transaction;
    int data;
    function void print();
        $display("write id=%0d data=%0d", id, data);
    endfunction
endclass
```

- **extends**: 자식 class가 부모의 property와 method를 상속한다.

`WriteTransaction`은 `id`, `data`, `print()`를 사용할 수 있지만, 부모 `Transaction`이 자식 고유의 `data`를 갖는 것은 아니다.

- **overriding**: 자식이 부모와 같은 method를 다시 정의한다. **부모 class의 구현 자체를 수정하지 않는다.**
- 일반적인 의미의 overloading은 같은 이름에 서로 다른 인자 구성을 두는 것이지만, SystemVerilog는 일반적인 method overloading을 지원하지 않는다.
- 부모 type의 handle은 자식 object를 가리킬 수 있다. 단, handle assignment가 object를 새로 생성하지는 않는다.

SystemVerilog / UVM

```

Transaction t;
WriteTransaction w;
w = new();
w.id = 3;
w.data = 42;
t = w;
t.print();

```

부모 print() 선언	method 선택 기준	위 코드의 t.print()
virtual 없음	호출에 사용한 handle type	부모 print()
virtual 있음	실제 object type	자식 print()

t의 handle type은 Transaction, 실제 object type은 WriteTransaction이다. 두 type을 구분하는 것이 polymorphism을 이해하는 핵심이다.

abstract class와 pure virtual method**SystemVerilog / UVM**

```

virtual class Transaction;
    pure virtual function void print();
endclass

class WriteTransaction extends Transaction;
    function void print();
        $display("write");
    endfunction
endclass

```

`pure virtual`은 부모에 method 구현을 두지 않고 구체적인 자식 class가 구현하도록 요구한다. 이를 선언하는 부모는 `virtual class`여야 하며 직접 `new()`로 생성할 수 없다. 부모 type의 handle 선언은 가능하고, 자식 object를 가리키게 한 뒤 `print()`를 호출할 수도 있다. 중간 자식도 `virtual class`로 남아 구현을 더 아래 자식에게 미룰 수 있다.

4. parameterized class

```
SystemVerilog / UVM
class Box #(type T = int);
    T value;
endclass

Box#(int)    numbers;
Box#(string) words;
Box          default_box; // T의 기본값 int 사용
```

type parameter를 두면 동일한 class 구조를 여러 데이터 type에 재사용할 수 있다. `T = int`는 `int`만 허용한다는 뜻이 아니라 type을 생략했을 때의 기본값이다. `Box#(int)`와 `Box#(string)`은 서로 다른 class type이므로 두 handle을 그대로 대입할 수 없다. `words.value = "12"` 같은 property 대입과 `words = numbers` 같은 handle 대입은 별개다.

5. static property와 static method

SystemVerilog / UVM

```
class Packet;
    static int count = 0;
    int id;

    function new();
        count++;
        this.id = count;
    endfunction

    static function int get_count();
        return count;
    endfunction
endclass
```

- `id`: 각 object마다 따로 존재하는 instance property.
- `static count`: class에서 공유하는 property. `Packet::count`로 접근할 수 있다.
- `static get_count()`: object 없이도 `Packet::get_count()`로 호출할 수 있다.
- handle 선언만으로는 constructor가 실행되지 않는다. `new()`가 실행될 때 이 예제의 `count++`와 `id = count`가 수행된다.
- 이미 저장된 `id`는 나중에 `count`를 수정해도 자동으로 바뀌지 않는다.

6. 접근 제한자

property 선언	같은 class	자식 class	class 밖
int data;	가능	가능	가능
protected int data;	가능	가능	불가능
local int data;	가능	불가능	불가능

외부에서 직접 수정하지 못하게 하려면 `local` 또는 `protected`로 제한하고 공개 method를 제공할 수 있다. 자식이 내부 property에 직접 접근해야 하는 설계라면 `protected`가 해당된다.

7. this와 super

```
SystemVerilog / UVM
class Transaction;
  int id;
  function new(int id);
    this.id = id;
  endfunction
endclass

class WriteTransaction extends Transaction;
  int data;
  function new(int id, int data);
    super.new(id);
    this.data = data;
  endfunction
endclass
```

- **this**: 현재 method가 실행 중인 object 자신. `this.data = data`에서 왼쪽은 object의 property, 오른쪽은 argument.
- **super**: "부모 object"가 아니라 부모 class에 정의된 member를 명시적으로 참조할 때 사용한다.

`super.new(id)`는 부모 constructor를 호출해 부모가 정한 초기화 절차를 재사용한다.

- 자식 object는 부모의 id를 상속받아 가지고 있다. `super.new(id)`를 쓰는 이유는 자식에게 id가 없어서가 아니다.

자주 헷갈렸던 지점

1. handle과 object는 다르다. `Packet p;`만으로 object가 생기지 않는다.
2. shallow copy는 중첩 handle을 공유한다. 단순 값 복사와 handle 대입을 구분한다.
3. overriding은 부모 구현을 변경하지 않는다. 자식 구현을 추가하는 것이다.
4. **virtual** 호출은 실제 **object type**이 기준이다. 단지 서로 다른 object라는 사실만으로 호출 method가 달라지는 것은 아니다.
5. **super**는 상위 object가 아니라 부모 class의 method/constructor를 참조한다.

현재 이해도와 다음 단계

대화 중 코드 해석과 확인 문제는 대부분 정확히 해결했다. 현재는 0~5 척도에서 대략 **2~3(예제를 해석하고 도움을 받아 설명 가능)** 수준으로 평가한다. 직접 코드를 작성하고 simulator에서 디버깅하는 능력은 아직 평가하지 않았다.

복습 질문:

1. `a = new(); b = a;`에서 object는 몇 개이며 `b = a`는 무엇을 복사하는가?
2. `y = new x` 이후 `x.hdr`와 `y.hdr`가 같은 내부 object를 가리키는 이유는?
3. 부모 type handle이 자식 object를 가리킬 때 **virtual** 유무에 따라 method 선택 기준은 어떻게 달라지는가?

다음 학습 주제는 2단계의 packed/unpacked array와 SystemVerilog 데이터 구조다.

PDF 보충 - 객체를 사용하는 검증 코드의 관점

2026-10-07 보강. 아래 추가 항목은 자료를 읽어 정리한 내용이며, 개별 문제로 이해도를 다시 확인한 것은 아니다. 연결: 01.03.03 Class - Encapsulation, 책 74~84쪽; 01.03.05 Class - Inheritance, 책 95~104쪽. PDF 파일의 페이지 번호는 책 쪽수 + 11이다.

Class와 module의 생성·실행 차이

Module/interface instance는 elaboration에서 만들어진 HDL 구조이고, class object는 실행 중 constructor로 생성한다. Class 안의 task는 object를 만들었다고 자동 실행되지 않는다. 호출한 process에서 수행하며, 병렬 실행에는 fork 등 별도 process가 필요하다. Class에 module의 initial/always를 그대로 넣는 방식으로 실행을 구성하지 않는다.

Handle 배열도 element마다 생성이 필요하다

SystemVerilog / UVM

```
Packet packets[3]; // handle 3개: 각각 null
foreach (packets[i]) packets[i] = new();
```

Queue나 array에 class type을 담아도 저장되는 것은 handle이다. Container 대입·추가가 내부 object의 deep copy를 의미하지는 않는다. 이 규칙은 mailbox, UVM analysis 통신과 7단계의 copy/clone에도 이어진다.

Downcast와 \$cast

부모 handle로 자식 object를 참조할 수 있지만, 자식 고유 field를 사용할 때는 해당 type의 handle이 필요하다. \$cast(child, parent)는 실제 object가 호환되는 type인지 확인하고 handle을 대입한다. 새로운 object나 복사본을 만들지 않는다. 반환값을 확인한 뒤 child에 접근한다.

Const, forward declaration, extern

Instance constant는 constructor에서 초기화해 object마다 다른 고정값을 갖게 할 수 있다. typedef class Header;는 뒤에서 정의할 class type을 먼저 알리는 선언이며 object를 생성하지 않는다. extern은 method 선언과 구현 위치를 나누는 문법으로, factory 등록과는 별개의 역할이다. 이 세 항목은 PDF 보충으로 읽은 범위다.

7단계와 연결

Method overriding은 자식의 method 구현을 선택하는 문제이고, factory override는 생성할 object type을 선택하는 문제다. 두 경우 모두 handle type과 실제 object type을 구분해야 한다. `super.do_compare()`는 같은 두 객체의 부모 구현을 호출해 상속된 field를 비교하는 것이며, 별도의 부모 객체를 비교하는 것이 아니다.

보충 확인 문제: `Packet q[$]; q.push_back(p); p.data = 9;`에서 `q[0].data`가 함께 바뀌는 이유는 무엇인가? 새 보충 문제이며 답변·실행은 아직 확인하지 않았다.

SystemVerilog/UVM 2단계 - 데이터 구조와 Process 통신



class type을 전달하면 handle이 이동한다. Mailbox 전달 자체가 object 복사는 아니다.

학습 범위: packed/unpacked array, fixed/dynamic array, queue, associative array, enum, struct, typedef, process/thread, fork-join, event, semaphore, mailbox. 코드는 개념 설명용이며 simulator에서 실행해 검증한 결과는 아니다.

1. Packed array와 unpacked array

Dimension이 변수 이름 **앞**에 있으면 packed, **뒤**에 있으면 unpacked다.

SystemVerilog / UVM

```

bit [7:0] a;          // 8-bit packed vector
bit b [7:0];          // 1-bit element 8개인 unpacked array
bit [3:0][7:0] c;     // 32-bit packed array
bit [7:0] d [3:0];    // 8-bit packed element 4개인 unpacked array
  
```

SystemVerilog / UVM

```
logic [7:0] memory [0:15];
```

- memory: logic [7:0] element 16개
- memory[3]: type은 logic [7:0], 폭은 8bit
- memory[3][2]: 네 번째 element의 bit 2
- packed는 vector처럼 bit 연산과 slicing에 적합하다.
- unpacked는 여러 element를 배열로 관리하는 데 적합하다.

2. Fixed array와 dynamic array

SystemVerilog / UVM

```
int fixed_array[4];  
int dynamic_array[];
```

- Fixed array: 선언할 때 element 수가 고정된다.
- Dynamic array: 선언 직후 크기는 0이며 runtime에 new[n]으로 할당한다.
- int는 element type이고, [4] 또는 new[4]는 element 수다. 폭과 array 크기를 혼동하지 않는다.

SystemVerilog / UVM

```
int values[];

values = new[3];
values[0] = 10;
values[1] = 20;
values[2] = 30;

values = new[5](values); // 기존 값을 보존하며 크기 변경
values.delete();        // size를 0으로 만들
```

- `new[5]`: 새 배열을 할당하며 기존 값 보존을 기대하면 안 된다.
- `new[5](values)`: 기존 값을 앞에서부터 복사하고 추가된 `int element`는 0으로 초기화한다.
- `values.size()`: 현재 `element` 수를 반환한다.

3. Queue

Queue는 양쪽 끝에서 `element`를 추가·제거하기 편한 가변 크기 자료구조다.

SystemVerilog / UVM

```
int q[$];

q.push_back(10); // {10}
q.push_back(20); // {10, 20}
q.push_front(5); // {5, 10, 20}

int a = q.pop_front(); // a=5, q={10,20}
int b = q.pop_back();  // b=20, q={10}
```

- `q[0]`: 첫 `element`를 제거하지 않고 읽는다.

- `q[$]`: 마지막 element를 제거하지 않고 읽는다.
- `pop_*`()은 값을 반환하면서 queue에서 제거한다.
- Scoreboard의 in-order expected transaction 대기열 등에 적합하다.

4. Associative array

사용한 key에 대해서만 element가 존재하는 sparse 자료구조다.

SystemVerilog / UVM

```
int score_by_id[int];          // value type=int, key type=int
int score_by_name[string];    // value type=int, key type=string

score_by_id[10] = 95;
score_by_id[42] = 80;
```

주요 method:

SystemVerilog / UVM

```
score_by_id.exists(10); // key 존재 여부
score_by_id.num();      // element 수
score_by_id.delete(10); // 특정 key 삭제
score_by_id.delete();   // 전체 삭제
```

기존 key에 다시 대입하면 element 수가 늘지 않고 값만 덮어쓴다. Transaction ID를 이용하는 out-of-order scoreboard matching 등에 적합하다.

5. Enum, typedef, struct

Enum과 typedef

SystemVerilog / UVM

```
typedef enum logic [1:0] {  
    IDLE   = 2'b00,  
    READ  = 2'b01,  
    WRITE = 2'b10,  
    ERROR = 2'b11  
} state_t;
```

- enum: 서로 관련된 named value 집합을 하나의 type으로 정의한다.
- typedef: 그 type에 재사용할 이름 `state_t`를 부여한다.
- `state.name()`으로 현재 enum 이름을 문자열로 얻을 수 있다.
- `parameter`도 값에 이름을 붙이지만, enum은 named value의 집합과 type을 만든다는 점이 다르다.

Struct

SystemVerilog / UVM

```
typedef struct {  
    bit [7:0]  address;  
    bit [31:0] data;  
    bit        write;  
} bus_item_t;
```

Struct는 관련 field를 하나의 value로 묶는다. Struct variable 대입은 class handle assignment와 달리 field 값 복사다.

SystemVerilog / UVM

```
bus_item_t a, b;
b = a;
b.data = 32'h1234; // a.data에는 영향 없음
```

struct packed은 모든 field를 연속된 packed vector처럼 배치한다.

SystemVerilog / UVM

```
typedef struct packed {
    bit [3:0] opcode;
    bit [7:0] address;
    bit [15:0] data;
} packet_t; // 총 28bit
```

선언 순서대로 opcode가 MSB 쪽, data가 LSB 쪽에 배치된다.

6. Process, thread, fork-join

fork 안의 각 statement는 별도의 child thread로 동시 진행된다.

문법	Parent가 기다리는 조건	남은 child thread
join	모든 child 종료	없음
join_any	하나의 child 종료	계속 실행
join_none	기다리지 않음	계속 실행
wait fork	모든 child 종료	완료될 때까지 대기

문법	Parent가 기다리는 조건	남은 child thread
disable fork	기다림 기능이 아님	활성 child를 종료

join_any는 남은 thread를 자동으로 종료하지 않는다.

SystemVerilog / UVM

fork

 forever begin

 #10;

 monitor();

 end

begin

 #105;

end

join_any

disable fork; // forever monitor 종료

join_none으로 생성한 child는 parent가 처음 멈추거나 종료될 때 실행을 시작한다. Zero-time 코드의 순서를 분석할 때 주의한다.

7. Event

Event는 데이터를 저장하지 않고 어떤 일이 발생했다는 사실만 알린다.

SystemVerilog / UVM

event done;

-> done; // trigger

@done; // 다음 trigger 대기

@done이 대기를 시작하기 전에 trigger가 이미 지나가면 event가 유실될 수 있다. Event는 과거 trigger 횟수를 queue처럼 기억하지 않는다. done.triggered는 같은 time slot의 race 완화에는 도움이 되지만 이전 simulation time의 trigger를 복구하지는 않는다.

8. Semaphore

Semaphore는 제한된 수의 key로 공유 자원 접근을 제어한다.

SystemVerilog / UVM

```
semaphore bus_lock = new(1);
```

```
bus_lock.get(1); // key가 없으면 기다림
```

```
// shared bus 사용
```

```
bus_lock.put(1); // key 반환
```

- get(n): blocking. key가 충분할 때까지 기다린다.
- try_get(n): nonblocking. 성공하면 1, 실패하면 0을 즉시 반환한다.
- put() 누락 시 다른 thread가 계속 기다리는 deadlock이 발생할 수 있다.
- 동일 시점에 경쟁하는 thread 중 누가 먼저 key를 얻는지는 코드만으로 단정하지 않는다.

9. Mailbox

Mailbox는 producer와 consumer 사이에서 데이터를 안전하게 전달하는 synchronized communication 수단이다.

SystemVerilog / UVM

```
mailbox #(int) mbx = new(2); // int 전용, 용량 2
```

Method	빈 상태 또는 가득 찬 상태	값 제거
put()	가득 차면 기다림	해당 없음

Method	빈 상태 또는 가득 찬 상태	값 제거
try_put()	가득 차면 즉시 0	해당 없음
get()	비면 기다림	제거함
try_get()	비면 즉시 0	성공 시 제거
peek()	비면 기다림	제거하지 않음
try_peek()	비면 즉시 0	제거하지 않음

mailbox #(int)에는 int만 넣을 수 있다. peek()과 try_peek()의 차이는 제거 여부가 아니라 blocking 여부다.

10. 도구 선택 기준

상황	적합한 도구
한 component 안에서 순서대로 저장·제거	Queue
Process 사이 producer-consumer 데이터 전달	Mailbox
발생 사실만 알림	Event
공유 자원의 동시 접근 제한	Semaphore
연속되지 않은 ID 기반 조회	Associative array

Shared variable이나 queue만 여러 thread가 함께 사용하면 대기·깨우기·동시 접근 정책을 직접 구현해야 한다. Mailbox와 semaphore는 이 동기화 의도를 명시적으로 표현한다.

자주 틀렸던 부분

1. logic [7:0] mem [0:15]에서 mem[3]은 16개 array가 아니라 8bit element 하나다.
2. Struct의 field를 enum 값으로 오해하지 않는다. Field는 선언된 자기 type의 값을 저장한다.
3. peek()도 mailbox가 비면 기다리는 blocking method다.
4. try_get()과 try_put()의 반환값은 실제 획득/저장 성공 여부다.
5. join_any는 남은 child를 종료하지 않는다. 필요한 경우 disable fork를 사용한다.

현재 이해도와 다음 단계

- 배열과 자료구조: 2~3/5
- enum, struct, typedef: 2/5
- process와 fork-join: 3/5
- event, semaphore, mailbox: 2~3/5

다음 학습 주제는 3단계 Randomization과 constraint다.

PDF 보충 - Container의 element와 process 수명

2026-10-07 보강. 추가 내용은 자료 보충이며 작성·실행 능력은 미평가다. 연결: 01.02.04~01.02.06, 책 36~45쪽; 01.03.01, 책 60~63쪽; 01.03.06, 책 105~114쪽.

foreach와 array method

foreach는 실제 index를 따라 element를 처리한다. find()는 조건에 맞는 element들의 queue를 반환하고, find_index()는 index들의 queue를 반환한다. sort()는 순서를 바꾸며 associative array에 같은 방식으로 적용하지 않는다. sum() with (int'(item))처럼 expression의 폭을 넓히면 작은 element type의 산술 폭 문제를 피할 수 있다.

SystemVerilog / UVM

```
int values[$] = {3, 8, 5};
int large[$];
large = values.find() with (item > 4);
values.sort();
```

Streaming operator는 bit열을 정한 순서와 chunk 크기로 묶거나 풀 때 사용한다. UVM의 `pack()`은 object field와 packer 정책을 사용하므로, SystemVerilog streaming 구문과 같은 API로 취급하지 않는다.

Function/task와 순차·동시 실행

Function은 시간 소비가 불가능하고 task는 가능하다. 일반적인 task 호출은 호출한 process에서 순차 실행된다. 10ns task 뒤에 20ns task를 호출하면 30ns, 두 task를 `fork...join`으로 동시에 시작하면 20ns가 걸리는 예로 8단계에서 확인했다. Simulator의 논리적 동시 실행과 CPU core 수는 별개의 문제다.

`disable fork`는 가장 가까운 fork 블록 이름만 기준으로 종료하는 문법이 아니다. 호출한 process의 활성 자손 process에 영향을 줄 수 있으므로, 종료할 작업을 별도 parent process 아래에 묶어 범위를 제어한다. 형제 child의 실행 순서는 의존하지 않는다.

Event의 발생과 완료 상태

`wait(done.triggered)`는 현재 time slot에서 발생한 trigger를 볼 수 있지만 과거 발생을 계속 저장하지 않는다. 반복 완료 확인에는 지속되는 `bit done_flag`나 mailbox처럼 의도에 맞는 상태/데이터 수단을 사용한다. `if`는 확인 시점에 한 번 분기하고 `wait(condition)`은 참이 될 때까지 process를 대기시킨다. 이미 참이면 즉시 통과한다.

Mailbox가 class object를 전달하는 경우**SystemVerilog / UVM**

```
mailbox #(Packet) mbx = new();
Packet sent, received;
sent = new();
mbx.put(sent);
mbx.get(received); // 같은 object의 handle을 전달
```

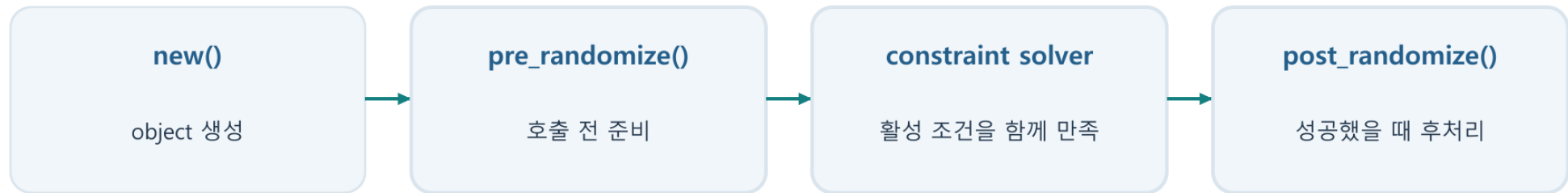
동기화된 전달과 object 복사는 별개의 일이다. Producer가 전달 후 같은 object를 수정하면 consumer가 보는 값도 바뀔 수 있다. 독립 snapshot이 필요하다면 복사 정책을 정한다.

PDF의 try_get()/try_peek() 예제에 보이는 숫자를 mailbox 길이로 일반화하지 않는다. Parameterized mailbox에서 반환값은 성공 여부를 판정하는 status로 사용한다. num()은 순간 길이이며 num()>0 확인 후 get() 사이에 다른 consumer가 제거할 수 있으므로 atomic한 try_get()과 같지 않다.

보충 확인 문제: mailbox에 handle을 put한 뒤 원본 field를 바꾸면 수신 객체에도 영향이 있는가? 새 보충 문제이며 답변·실행은 아직 확인하지 않았다.

SystemVerilog/UVM 3단계 - Randomization과 Constraint

randomize() 호출



new()는 자동 randomization이 아니다. randomize() 반환값으로 성공 여부를 확인한다.

size + element 범위 + 합계 조건의 모순을 먼저 확인

학습 범위: rand, randc, randomize(), constraint block, inline constraint, inside, dist, implication, conditional constraint, solve before, soft, constraint_mode(), rand_mode(), pre_randomize(), post_randomize(), constraint 상속과 override, 충돌 및 과도하거나 부족한 제약.

1. Randomization의 기본 구조

SystemVerilog / UVM

```
class Packet;
    rand bit [3:0] addr;
    bit [3:0] data;
endclass

Packet p = new();
p.addr = 3;
p.data = 5;

if (!p.randomize())
    $error("Randomization failed");
```

- `new()`는 object를 생성한다.
- `randomize()`는 활성화된 `rand`와 `randc` field에 constraint를 만족하는 값을 실제로 반영한다.
- `rand`가 없는 `data`는 randomization 대상이 아니므로 기존값을 유지한다.
- `randomize()`는 성공 시 1, 실패 시 0을 반환한다. 반환값을 무시하지 않는다.
- Randomization이 실패하면 field는 이전값을 유지한다. 이 값을 정상 transaction으로 사용하면 안 된다.

2. rand와 randc

SystemVerilog / UVM

```
rand bit [1:0] a;
randc bit [1:0] b;
```

- `rand`: 매 호출에서 합법적인 값 하나를 선택하며 같은 값이 연속으로 나올 수 있다.
- `randc`: random-cyclic 방식이다. 한 주기 안에서 가능한 값을 한 번씩 사용한 뒤 새 주기를 시작한다.

- randc도 주기 경계에서는 같은 값이 연속될 수 있다.

개념 흐름

첫 주기: 2, 0, 3, 1

둘째 주기: 1, 3, 0, 2

3. Constraint는 대입문이 아니라 조건이다

SystemVerilog / UVM

```
constraint c_addr {
    addr >= 4;
    addr <= 9;
}
```

Constraint block 안의 식은 기본적으로 AND 관계다.

개념 흐름

(addr >= 4) AND (addr <= 9)

가능한 값: 4, 5, 6, 7, 8, 9

다음처럼 서로 모순되면 해가 없다.

SystemVerilog / UVM

```
constraint c_bad {
    addr >= 4;
    addr < 3;
}
```

Solver는 활성화된 모든 조건을 동시에 만족하는 조합을 찾는다. 코드를 위에서 아래로 실행하며 값을 순서대로 대입하는 방식이 아니다.

4. Inline constraint와 inside

SystemVerilog / UVM

```
if (!p.randomize() with {  
    addr inside {[2:5], 9, [12:14]};  
})  
    $error("Randomization failed");
```

가능한 값은 2, 3, 4, 5, 9, 12, 13, 14다.

SystemVerilog / UVM

```
!(addr inside {[2:5], 9, [12:14]});
```

addr가 4bit라면 가능한 값은 0, 1, 6, 7, 8, 10, 11, 15다.

Inline constraint는 기존 class constraint를 자동으로 대체하지 않는다. 보통 기존 조건에 AND로 추가되며 해당 randomize() 호출에만 적용된다.

5. dist와 가중치

SystemVerilog / UVM

```
mode dist {  
    0 := 1,  
    1 := 3  
};
```

가중치 합이 4이므로 0은 25%, 1은 75%의 확률을 갖는다. 네 번 호출한다고 반드시 1회와 3회로 나뉘는 것은 아니다.

범위에서는 :=와 ./의 차이가 중요하다.

SystemVerilog / UVM

```
value dist {
    0      := 10,
    [1:3] := 30
};
```

:=는 범위의 각 값에 weight를 적용한다. Weight는 10, 30, 30, 30이고 확률은 10%, 30%, 30%, 30%다.

SystemVerilog / UVM

```
value dist {
    0      :/ 10,
    [1:3] :/ 30
};
```

:/는 범위 전체가 weight를 나눠 갖는다. [1:3]의 각 값은 weight 10을 가지므로 네 값의 확률은 각각 25%다.

6. Implication과 conditional constraint

SystemVerilog / UVM

```
write == 1 -> addr inside {[8:15]};
```

- `write == 1`이면 오른쪽 조건을 반드시 만족한다.
- `write == 0`이면 implication 전체가 참이 되며 이 식은 `addr`를 제한하지 않는다.

양쪽 경우를 모두 제한하려면 conditional constraint를 사용한다.

SystemVerilog / UVM

```

if (write)
    addr inside {[8:15]};
else
    addr inside {[0:3]};

```

7. solve before

SystemVerilog / UVM

```

constraint c_value {
    (kind == 0) -> length == 0;
    (kind == 1) -> length inside {[1:3]};
    solve kind before length;
}

```

`solve before`는 합법적인 조합을 바꾸지 않고 분포에 영향을 준다. 위 예제에서는 `kind`를 먼저 선택하므로 `kind=0`과 `kind=1`이 각각 약 50%가 된다. 이후 선택된 `kind` 안에서 가능한 `length`가 나뉜다.

개념 흐름

```

P(kind=0, length=0) = 1/2
P(kind=1, length=1) = 1/2 × 1/3 = 1/6
P(kind=1, length=2) = 1/6
P(kind=1, length=3) = 1/6

```

`solve before`는 절차적 실행 순서도 아니고 값의 대입 순서도 아니다.

8. soft constraint

```
SystemVerilog / UVM
constraint c_default {
    soft addr inside {[0:7]};
}
```

soft는 기본 조건이다. 다른 조건이 생겼다고 즉시 사라지는 것이 아니라, 함께 만족할 수 없을 때 우선순위가 높은 조건에 양보한다.

```
SystemVerilog / UVM
p.randomize() with { addr inside {[4:11]}; };
```

두 조건의 교집합이 있으므로 soft constraint는 유지되고 최종 범위는 4~7이다. 교집합이 전혀 없다면 hard inline constraint가 유지되고 soft constraint가 제거된다. Class soft와 inline soft가 충돌하면 inline soft가 우선한다.

9. rand_mode와 constraint_mode

```
SystemVerilog / UVM
p.addr.rand_mode(0);           // addr 값을 새로 뽑지 않음
p.c_addr.constraint_mode(0);   // c_addr 조건을 검사하지 않음
```

두 메서드는 서로 독립적이다. rand_mode(0)으로 field를 고정해도 그 field를 참조하는 constraint는 자동으로 꺼지지 않는다.

```
SystemVerilog / UVM
p.addr = 12;
p.addr.rand_mode(0);

constraint c_addr {
    addr inside {[0:3]};
}
```

Solver는 `addr`를 변경할 수 없으며 고정된 12가 조건을 위반하므로 randomization은 실패한다. 실패 후 `addr`는 12를 유지한다.

개념 흐름

`rand` 활성화 → 현재값을 solver가 변경할 수 있음

`rand` 비활성화 → 현재값이 고정되며 constraint 검사에는 계속 참여함

10. `pre_randomize`와 `post_randomize`

SystemVerilog / UVM

```
function void pre_randomize();
    // 랜덤화 전 mode나 constraint 상태 조정
endfunction

function void post_randomize();
    // 결정된 값으로 파생값 또는 checksum 계산
endfunction
```

호출 흐름은 다음과 같다.

개념 흐름

`randomize()` 호출

→ `pre_randomize()`

→ constraint solving 및 값 반영

→ 성공 시 `post_randomize()`

→ 반환

`pre_randomize()`에서 `rand` field에 값을 대입해도 `rand_mode`가 켜져 있으면 solver가 그 값을 다시 바꿀 수 있다. 두 hook은 function이므로 simulation time을 소비할 수 없다.

11. Constraint 상속과 override

부모와 자식의 constraint 이름이 다르면 두 조건이 모두 적용된다.

SystemVerilog / UVM

```
class BasePacket;
    rand bit [3:0] addr;
    constraint c_range { addr inside {[2:10]}; }
endclass

class PacketA extends BasePacket;
    constraint c_even { addr % 2 == 0; }
endclass
```

최종 가능한 값은 2, 4, 6, 8, 10이다.

자식 class에서 부모와 같은 이름의 constraint block을 선언하면 부모 constraint를 override한다.

SystemVerilog / UVM

```
class PacketB extends BasePacket;
    constraint c_range { addr inside {[12:15]}; }
endclass
```

최종 가능한 값은 12, 13, 14, 15다.

12. Conflicting, over-constrained, under-constrained

상태	의미	결과 또는 위험
Conflicting	조건이 모순되어 해가 없음	randomize() 실패
Over-constrained	합법적인 상황을 지나치게 제한	성공할 수 있지만 검증 다양성 감소
Under-constrained	필요한 조건이 빠짐	합법 범위 밖의 값이나 의도하지 않은 조합 생성

Over-constrained가 반드시 실패를 뜻하는 것은 아니다. 예를 들어 사양상 1~8이 모두 합법인데 `length == 4`만 허용하면 해는 있지만 다른 합법적 경우를 검증하지 못한다.

13. 실패를 안전하게 처리하기

SystemVerilog / UVM

```
if (!item.randomize())
  `uvm_fatal("RANDFAIL", "Item randomization failed")
```

반환값을 무시하면 실패 이전의 값이 driver로 전달될 수 있다. Constraint를 작성한 뒤에는 다음을 확인한다.

1. 값 공간이 의도한 범위인가?
2. 모든 조건을 동시에 만족하는 해가 있는가?
3. 분포가 의도와 일치하는가?
4. 필요 이상으로 합법적 경우를 제외하지 않았는가?
5. 실패를 즉시 보고하고 transaction 전달을 중단하는가?

자주 틀렸던 부분

1. 활성화된 rand field의 초기값은 solver가 변경할 수 있다.
2. rand_mode(0)은 constraint를 고지 않는다. 고정된 현재값도 활성 constraint를 만족해야 한다.
3. Inline constraint는 기존 hard constraint를 자동으로 override하지 않는다.
4. soft는 다른 조건이 추가됐다는 이유만으로 사라지지 않는다. 교집합이 있으면 함께 적용된다.
5. solve before는 해의 집합이 아니라 분포를 바꾼다.
6. dist의 숫자는 횟수 보장이 아니라 상대적인 weight다.

현재 이해도와 다음 단계

- 기본 randomization과 반환값: 3/5
- inside, dist, implication, conditional: 2~3/5
- solve before, soft: 2/5
- rand_mode, constraint_mode, 실패 분석: 2~3/5
- Constraint 상속과 상태 분류: 2~3/5

다음 학습 주제는 4단계 Interface와 simulation timing이다.

PDF 보충 - Transaction의 값 공간 설계

2026-10-07 보강. 추가 항목은 PDF 보충으로 읽었으며 개별 이해도는 미확인이다. 연결: 01.03.04, 책 86~94쪽; 02.04 Transaction constraints/override, 책 168~171~172쪽.

생성과 randomization을 분리한다

PDF 책 87쪽의 new() 주석은 자동 randomization처럼 읽힐 수 있다. 일반적인 new()는 constructor를 실행하며 randomize()를 자동 호출하지 않는다. Constructor에 명시적으로 호출한 경우만 별도다. 성공 로그를 보지 않은 예제는 실행 통과로 판단하지 않는다.

Array의 크기와 각 element 조건

```
SystemVerilog / UVM
class burst_item;
  rand bit [7:0] payload[];
  constraint c_payload {
    payload.size() inside {[2:8]};
    foreach (payload[i]) payload[i] inside {[1:15]};
    payload.sum() with (int'(item)) <= 60;
  }
endclass
```

Size, element 범위, 전체 합은 동시에 만족해야 하는 조건이다. 합계를 구하는 expression의 폭을 명시하고, 합과 size가 서로 모순되는지 확인한다. `unique { ... }`는 서로 다른 값을 요구하는 보충 문법이며 `randc`의 호출 간 cycle과 구분한다.

Must-obey와 scenario rule

PDF의 분류는 사양상 항상 지킬 조건과 특정 test가 선택한 조건을 나눠 관리하라는 설계 관점이다. 별도의 언어 keyword가 아니다. Scenario별 constraint를 이름별로 나누면 `constraint_mode()`로 의도를 드러내기 쉽다. Illegal stimulus를 검증할 때는 어떤 규칙을 의도적으로 해제했는지 기록한다.

Factory가 constraint를 교체하는 연결

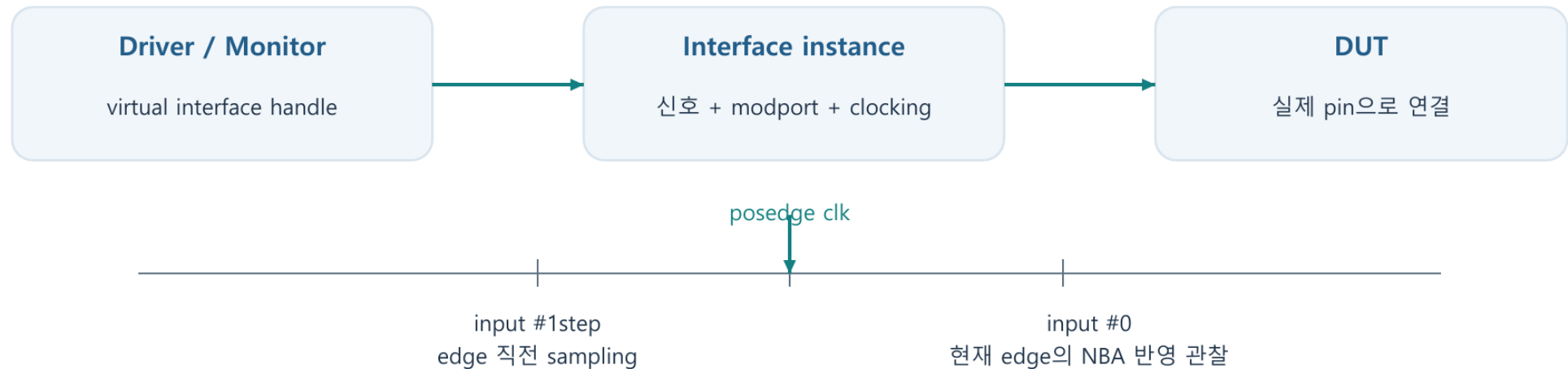
자식 transaction에 다른 이름의 constraint를 추가하면 부모의 조건과 함께 적용된다. 같은 이름이면 상속된 constraint를 대체한다. Factory override는 이런 자식 type을 생성하도록 선택하며, `randomize()`는 그 실제 object의 활성 constraint를 사용한다. Factory 자체가 rand field에 값을 넣는 것은 아니다.

Solver 안의 function과 재현성

Constraint 안의 function은 부작용 없는 계산으로 작성한다. 호출 횟수.순서를 로그 제어 수단으로 사용하거나 randomization mode를 함수 안에서 바꾸지 않는다. Seed, test 이름, simulator/UVM 버전, 실패한 constraint와 호출 위치를 함께 남겨 재현한다. 분포 설명은 값의 폭과 다른 활성 constraint를 명시한 예제에 한정한다.

보충 확인 문제: `payload.size()==8`이고 모든 element가 10 이상인데 `sum<=60`이면 해가 존재하는가? 새 보충 문제이며 답변·실행은 아직 확인하지 않았다.

SystemVerilog/UVM 4단계 - Interface와 Simulation Timing



순서 개념 그림: 시간 간격은 실제 축척이 아님. 신호별 sampling 정책은 protocol로 결정.

학습 범위: interface, interface instance, modport, virtual interface, clocking block, input/output skew, simulation event region, blocking/nonblocking assignment, race condition, driver 구동 시점, monitor sampling 시점, registered output 관찰.

1. Interface가 필요한 이유

여러 protocol 신호를 module port로 각각 전달하면 component가 늘어날수록 연결 코드가 반복된다. 신호 방향, clock 기준 동작, 공통 task도 여러 위치에 흩어질 수 있다.

Interface는 관련 신호와 timing 규칙을 하나의 구조로 묶는다.

SystemVerilog / UVM

```
interface bus_if(input logic clk);  
    logic    valid;  
    logic    ready;  
    logic [7:0] data;  
endinterface
```

Interface에는 다음 항목을 함께 둘 수 있다.

- Protocol signal
- modport
- clocking block
- 공통 task와 function

2. Interface type과 instance

SystemVerilog / UVM

```
interface bus_if(input logic clk);  
    logic    valid;  
    logic    ready;  
    logic [7:0] data;  
endinterface  
  
module top;  
    logic clk;  
    bus_if bus(.clk(clk));  
endmodule
```

`bus_if`는 interface의 구조를 정의한 type이다. `bus`는 top에서 실제로 생성된 interface instance다.

개념 흐름

bus_if → interface 설계도

bus → 실제 interface instance

clk는 top에서 생성한 clock을 interface가 입력으로 받는다. valid, ready, data는 interface 내부에 선언된 protocol signal이다.

3. DUT 연결 방식

DUT가 개별 port를 사용하면 interface signal을 각각 연결한다.

SystemVerilog / UVM

```
dut u_dut (
    .clk    (clk),
    .valid  (bus.valid),
    .ready  (bus.ready),
    .data   (bus.data)
);
```

DUT도 interface port를 받도록 작성하면 interface instance 하나로 연결할 수 있다.

SystemVerilog / UVM

```
module dut(bus_if bus);
    // ...
endmodule

dut u_dut(bus);
```

Interface의 목적은 port 수를 줄이는 것에만 있지 않다. Protocol의 신호, 역할, 구동 시점과 sampling 시점을 한곳에 정의하는 것이 더 중요한 목적이다.

4. Modport

modport는 interface를 사용하는 주체별로 신호 접근 방향을 정의한다. 방향은 interface의 절대적인 방향이 아니라 해당 modport를 사용하는 쪽에서 본 방향이다.

SystemVerilog / UVM

```
interface bus_if(input logic clk);
    logic    valid;
    logic    ready;
    logic [7:0] data;

    modport DUT (
        input  clk,
        input  valid,
        input  data,
        output ready
    );

    modport DRV (
        input  clk,
        input  ready,
        output valid,
        output data
    );

    modport MON (
        input clk,
        input valid,
        input ready,
        input data
    );
endinterface
```

Driver는 `valid`와 `data`를 구동하고 `ready`를 읽는다. Monitor는 모든 신호를 관찰하며 어떤 신호도 구동하지 않는다. DUT는 `valid`와 `data`를 입력받고 `ready`를 출력한다.

5. Virtual interface

실제 interface instance는 module과 함께 elaboration 과정에서 정적으로 생성된다. Class object는 simulation 중 `new()`로 동적으로 생성된다. Class 안에서는 실제 interface instance를 `new()`로 만들 수 없다.

Class는 virtual interface handle을 통해 top에서 생성된 실제 interface instance를 가리킨다.

```
SystemVerilog / UVM
class Driver;
    virtual bus_if.DRV vif;
endclass
```

각 부분의 의미는 다음과 같다.

```
개념 흐름
virtual → 실제 interface를 가리키는 handle
bus_if → interface type
.DRV → 사용할 modport
vif → 사용자가 정한 member variable 이름
```

`vif`는 SystemVerilog keyword가 아니다. Virtual interface의 줄임말로 자주 사용하는 관례적인 이름이다.

```
SystemVerilog / UVM
Driver drv = new();
drv.vif = bus;
```

이 대입은 interface를 복사하거나 새로 만들지 않는다. `drv.vif`가 실제 instance `bus`를 가리키게 한다.

6. Null virtual interface

Class object만 생성하고 vif를 연결하지 않으면 vif는 null이다.

SystemVerilog / UVM

```
Driver drv = new();

if (drv.vif == null)
    $fatal(1, "virtual interface is null");
```

Null 상태에서 vif.drv_cb나 vif.data에 접근하면 runtime 오류가 발생한다. 이는 신호가 단순히 전달되지 않는 상태가 아니라 유효한 interface instance가 전혀 연결되지 않은 상태다.

7. Clocking block

clocking block은 특정 clock event를 기준으로 testbench가 신호를 언제 구동하고 sampling할지 정의한다.

SystemVerilog / UVM

```
clocking drv_cb @(posedge clk);
    default input #1step output #0;
    output valid, data;
    input ready;
endclocking
```

drv_cb는 사용자가 정한 이름이다. 보통 driver용 clocking block이라는 의미로 사용한다.

SystemVerilog / UVM

```
@(vif.drv_cb);
vif.drv_cb.valid <= 1;
vif.drv_cb.data <= 8'h55;
```

@(vif.drv_cb)는 drv_cb에 지정된 posedge clk event를 기다린다. vif.drv_cb.valid는 별도의 가상 신호가 아니라 실제 interface instance의 valid에 대한 clocking block view다.

8. Modport와 clocking block의 차이

개념 흐름

modport → 누가 무엇을 읽고 쓸 수 있는가

clocking block → 언제 구동하고 sampling하는가

modport는 접근 역할과 방향을 구분한다. clocking block은 clock을 기준으로 접근 시점을 구분한다.

9. Race condition

DUT와 testbench가 같은 clock edge와 같은 simulation event region에서 같은 신호를 읽고 쓰면 실행 순서에 따라 결과가 달라질 수 있다.

```
SystemVerilog / UVM
// DUT
always_ff @(posedge clk) begin
    if (valid)
        captured_data <= data;
end

// Testbench
@(posedge clk);
valid = 1;
data  = 8'hA5;
```

같은 posedge에서 DUT와 testbench가 모두 깨어난다. Testbench가 먼저 실행되면 DUT가 새 값을 볼 수 있다. DUT가 먼저 실행되면 이전 값을 볼 수 있다. 이처럼 실행 순서에 따라 결과가 바뀌는 상황이 race condition이다.

10. Blocking과 nonblocking assignment

Blocking assignment는 statement가 실행되는 즉시 왼쪽 값을 갱신한다.

```
SystemVerilog / UVM  
data = 8'hA5;
```

Nonblocking assignment는 오른쪽 값을 현재 region에서 계산하고 왼쪽 값의 갱신을 NBA region에 예약한다.

```
SystemVerilog / UVM  
data <= 8'hA5;
```

개념 흐름

Active region → 오른쪽 값 계산, update 예약
NBA region → 왼쪽 값 실제 갱신

Nonblocking assignment를 사용하면 같은 Active region의 다른 process는 update 이전 값을 읽는다. 이는 실행 순서에 따른 불확실성을 줄이지만, 어느 cycle의 값을 관찰할지는 여전히 protocol에 맞게 설계해야 한다.

11. Driver의 구동 시점

Driver가 clocking block의 output #0으로 edge에서 값을 구동하면 DUT가 같은 edge에서 새 값을 받아야 하는 것으로 해석하지 않는다. Driver는 다음 sampling edge에서 사용할 값을 준비한다.

개념 흐름

5ns posedge

- └─ DUT는 기존의 안정된 입력을 sampling
- └─ Driver는 valid=1, data=A5 구동

5ns~15ns

- └─ valid=1, data=A5 안정적으로 유지

15ns posedge

- └─ DUT가 valid=1, data=A5 sampling

이 방식은 DUT와 driver가 같은 edge의 Active region에서 먼저 실행되려고 경쟁하는 상황을 피한다.

12. Monitor의 sampling 시점

DUT가 registered output을 nonblocking assignment로 갱신한다고 가정한다.

SystemVerilog / UVM

```
always_ff @(posedge clk)
    result <= input_data;
```

Edge 직전 result=10이고 input_data=20이라면 event 흐름은 다음과 같다.

개념 흐름

Edge 직전 → result=10

Active → result <= 20 update 예약

NBA → result=20 실제 반영

Observed → 갱신된 result 관찰 가능

13. input #1step과 input #0

SystemVerilog / UVM

```
clocking mon_before_cb @(posedge clk);
    input #1step result;
endclocking
```

input #1step은 clock edge 바로 직전의 안정된 값을 sampling한다. 위 예제에서는 이전값 10을 본다.

SystemVerilog / UVM

```
clocking mon_after_cb @(posedge clk);
    input #0 result;
endclocking
```

input #0은 같은 time slot에서 DUT의 NBA update가 반영된 뒤 값을 sampling한다. 위 예제에서는 갱신된 값 20을 본다.

개념 흐름

```
input #1step → edge 직전값
input #0      → NBA 반영 후 값
```

어느 방식이 옳은지는 protocol에서 정의한 관찰 시점에 따라 결정한다. Request와 input의 edge 직전 상태를 기록할 때는 #1step이 적합할 수 있다. 해당 edge에서 갱신된 registered output을 기록할 때는 #0을 고려할 수 있다.

14. 임의의 #1 delay가 위험한 이유

#1step은 simulator time precision의 한 단계로 edge 직전을 의미한다. #1은 현재 timeunit을 기준으로 실제 simulation time을 1만큼 지연한다.

Race를 피하려고 monitor에 임의의 #1 delay를 넣으면 clock 주거나 timeunit이 변경될 때 sampling 위치도 달라진다. 이 방식은 timing 문제를 해결하지 않고 우연히 숨길 수 있다. Clocking block과 명확한 skew를 사용해 의도한 sampling 시점을 표현하는 편이 안전하다.

15. 통합 예제

SystemVerilog / UVM

```
interface bus_if(input logic clk);
    logic        valid;
    logic        ready;
    logic [7:0] data;
    logic [7:0] result;

    clocking drv_cb @(posedge clk);
        default input #1step output #0;
        output valid, data;
        input  ready;
    endclocking

    clocking mon_cb @(posedge clk);
        input #0 valid, ready, data, result;
    endclocking

    modport DRV (clocking drv_cb);
    modport MON (clocking mon_cb);
endinterface

class Driver;
    virtual bus_if.DRV vif;
endclass

class Monitor;
    virtual bus_if.MON vif;
endclass
```

Driver는 **valid**와 **data**를 구동하고 **ready**를 관찰한다. Monitor는 모든 신호를 관찰하며 아무 신호도 구동하지 않는다. **mon_cb**의 **input #0**은 DUT의 NBA update가 반영된 registered output을 sampling할 수 있다.

자주 틀렸던 부분

1. `bus_if`는 type이고 `bus`는 실제 instance다.
2. `vif`는 keyword가 아니라 virtual interface handle에 자주 사용하는 변수 이름이다.
3. Virtual interface는 별도의 신호를 만들지 않고 실제 interface instance를 가리킨다.
4. Null `vif`는 단순한 데이터 미수신이 아니라 runtime 접근 오류다.
5. `modport`는 접근 방향을, `clocking block`은 접근 시점을 정의한다.
6. `input #1step`은 edge 직전값을, `input #0`은 NBA 반영 후 값을 sampling한다.
7. Driver가 현재 edge에서 구동한 값은 다음 sampling edge를 위한 값으로 이해한다.

현재 이해도와 다음 단계

- Interface type과 instance: 3/5
- Modport: 3/5
- Virtual interface와 null 처리: 2~3/5
- Clocking block과 race condition: 2~3/5
- Event region과 registered output sampling: 2/5

다음 학습 주제는 5단계 UVM 개요와 전체 구조다.

PDF 보충 - Parameter와 sampling 기준의 일치

2026-10-07 보강. Parameterized interface와 전달 방식은 자료 보충이며 실제 연결·실행은 미확인이다. 연결: 01.03.02 Interface, 책 64~73쪽; 01.03.05 Virtual interfaces, 책 103쪽; 02.03-02.12의 top/interface 연결 예.

Parameter와 modport도 type의 일부다

SystemVerilog / UVM

```
interface bus_if #(parameter int DW = 8)(input logic clk);
    logic [DW-1:0] data;
endinterface
// class 안의 선언 예
virtual bus_if#(16) vif;
```

실제 instance, virtual interface 선언, 이후 config DB의 type parameter는 꼭과 modport까지 일치시킨다. Config DB 전달 상세는 10단계에서 확인한다. Constructor를 통한 직접 전달도 가능하며 interface를 복사하지 않고 실제 instance에 대한 참조를 전달한다.

Clocking input skew와 output skew

Clocking block의 input은 관찰하는 쪽, output은 구동하는 쪽이다. 일반적인 양의 input skew는 edge보다 앞서 sampling하고, 양의 output skew는 edge 뒤에 구동한다. 명시하지 않은 기본값은 input #1step, output #0이다. Clocking block의 input #0과 절차문 #0;은 같은 동작이 아니다. 절차문 #0만으로 NBA 이후 관찰을 보장하지 않는다.

한 monitor에서 요청과 결과의 시점을 구분

기존 통합 예제의 모든 신호 input #0은 출력 갱신 관찰을 설명한 예다. 그대로 사용하면 driver가 edge 뒤에 갱신한 입력을 DUT가 그 edge에서 사용한 입력으로 오해할 수 있다. 실제 protocol monitor는 요청 수락 시점의 입력과 등록 출력 갱신 시점을 각각 정한다.

SystemVerilog / UVM

```
clocking mon_cb @(posedge clk);
    input #1step valid, ready, data;
    input #0 result;
endclocking
```

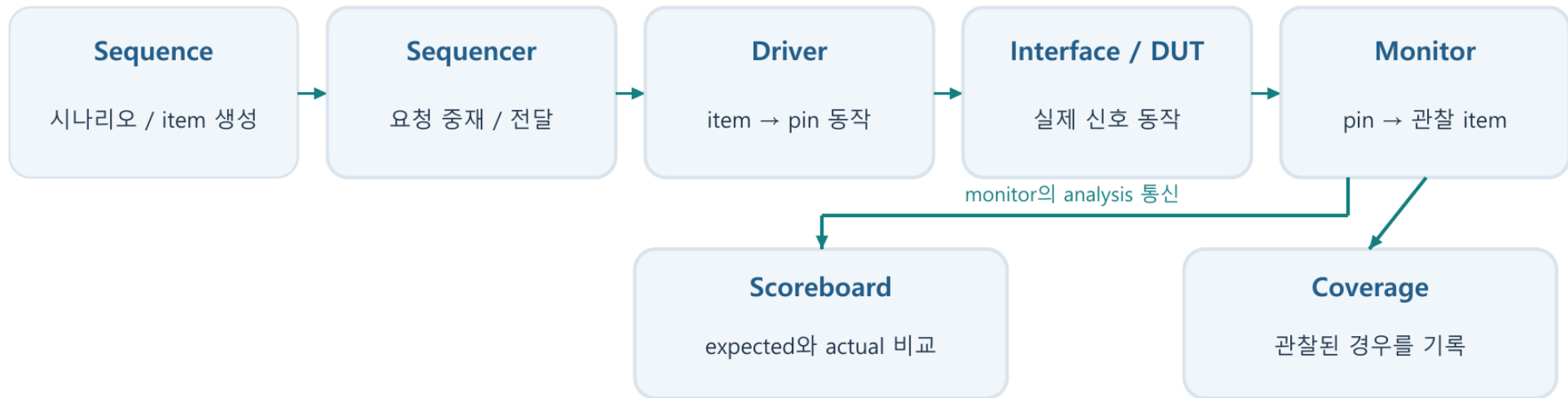
이 예도 protocol에서 ready와 result의 의미를 확인해 선택해야 한다. FIFO에서는 wr_en/rd_en, full/empty의 edge 직전 상태로 수락을 판단하고 해당 edge 뒤의 등록 출력은 별도 timing으로 연결할 수 있다. 구체 정책은 DUT RTL을 읽은 뒤 정한다.

Program은 UVM의 필수 조건이 아니다

PDF의 program block은 Reactive region을 활용하는 설명 방식이다. UVM top을 module로 작성해 run_test()를 호출할 수도 있다. 어느 방식이든 pin 접근 timing과 clocking block 정책을 명시해야 한다. NBA, Observed, Reactive/Re-NBA 등 실제 scheduling은 4단계의 단순 그림보다 세부적이다.

보충 확인 문제: input #0 result와 절차문 #0 뒤 result 읽기는 왜 같다고 단정할 수 없는가? 새 보충 문제이며 답변·실행은 아직 확인하지 않았다.

SystemVerilog/UVM 5단계 - UVM 개요와 전체 구조



이 그림은 데이터 흐름이다. Component hierarchy와 실행 호출 순서는 별도로 본다.

학습 범위: UVM의 필요성, directed verification, constrained-random verification, transaction-level verification, reuse, test, environment, agent, active/passive agent, sequencer, driver, monitor, analysis port, reference model, scoreboard, subscriber, virtual sequence와 virtual sequencer.

1. UVM이 필요한 이유

작은 testbench에서는 하나의 **initial** block에서 입력 생성, pin 구동, 출력 관찰, 결과 비교를 모두 수행할 수 있다. 검증 규모가 커지면 stimulus, protocol timing, checking, coverage가 뒤섞이고 test마다 같은 코드가 반복된다.

UVM은 책임을 나누고 transaction 단위로 연결하여 검증환경을 재사용할 수 있게 한다.

개념 흐름

검증 의도와 순서	→ sequence
transaction 전달 중재	→ sequencer
pin-level 구동	→ driver
실제 pin 관찰	→ monitor
expected 계산	→ reference model
expected/actual 비교	→ scoreboard
검증 범위 측정	→ coverage subscriber

2. Directed와 constrained-random verification

Directed verification은 검증자가 시험할 값을 직접 지정한다.

SystemVerilog / UVM

```
item.addr = 8'h10;
item.data = 8'h55;
item.write = 1;
```

특정 기능, 경계값, 이미 발견한 버그를 정확히 재현하기 좋다. 다만 검증자가 생각한 경우만 시험하기 쉽다.

Constrained-random verification은 합법적인 값 공간을 constraint로 정의하고 그 안에서 다양한 값을 생성한다.

SystemVerilog / UVM

```
assert(item.randomize() with {
    addr inside {[8'h10:8'h1F]};
    write == 1;
});
```

예상하지 못한 조합을 넓게 탐색할 수 있지만, 중요한 경우가 실제로 생성됐는지는 coverage로 확인해야 한다.

randomize()를 사용했는지만으로 두 방식을 구분하지 않는다. 모든 field를 하나의 값으로 고정했다면 문법은 randomization이어도 검증 의도는 directed에 가깝다.

개념 흐름

Directed → 정확한 경우 지정

Constrained-random → 합법 공간을 정의하고 여러 조합 탐색

실제 검증에서는 두 방식을 함께 사용한다.

3. Transaction-level verification

Transaction은 여러 pin 변화로 이루어진 protocol 동작을 의미 있는 작업 하나로 표현한다.

SystemVerilog / UVM

```
item.write = 1;
item.addr  = 8'h20;
item.data  = 8'hA5;
```

개념 흐름

Transaction level: 0x20 주소에 0xA5를 쓴다.

Pin level: valid를 올리고, addr/data를 구동하고, ready를 기다린다.

Sequence는 무엇을 보낼지 결정하고 driver는 그 작업을 protocol signal과 timing으로 변환한다. Protocol timing이 변경돼도 sequence를 유지하고 driver를 중심으로 수정할 수 있으므로 재사용성이 높아진다.

Sequence가 virtual interface를 직접 사용하면 signal 이름과 timing에 결합되어 재사용성이 낮아진다.

4. UVM 전체 흐름

개념 흐름

test

→ sequence

→ sequencer

→ driver

→ interface

→ DUT

→ monitor

→ analysis port

→ scoreboard / coverage

각 화살표가 모두 parent-child 생성을 뜻하지는 않는다. 위치에 따라 sequence 시작, transaction 중재, signal 변환, 관찰 결과 방송을 의미한다.

개념 흐름

Test

→ 환경과 시나리오를 선택한다.

Sequence

→ transaction을 생성하고 순서를 정한다.

Sequencer

→ sequence와 driver 사이 전달을 중재한다.

Driver

→ transaction을 signal로 변환하고 구동한다.

Interface

→ class 세계와 DUT signal을 연결한다.

DUT

→ 설계 기능을 수행한다.

Monitor

→ signal을 관찰하고 transaction으로 복원한다.

Analysis port

→ 관찰 transaction을 방송한다.

Scoreboard

→ expected와 actual을 비교한다.

Coverage

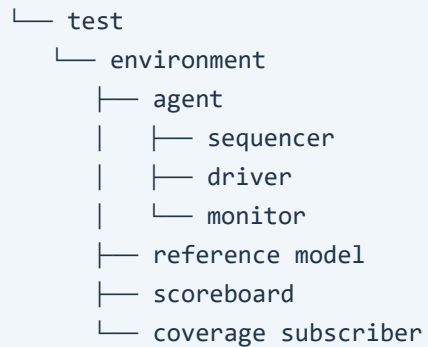
→ 어떤 값과 조합이 검증됐는지 측정한다.

5. Test, environment, agent

일반적인 구성과 생성 관계는 다음과 같다.

개념 흐름

UVM



- Test: 이번 simulation의 environment, 설정, 실행할 sequence를 선택한다.
- Environment: agent, scoreboard 등 검증 구성 요소를 묶고 연결한다.
- Agent: 특정 interface 또는 protocol 하나의 구동과 관찰 요소를 묶는다.

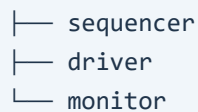
Test, environment, agent는 구조와 구성을 담당한다. 실제 pin-level 구동과 관찰은 driver와 monitor가 담당한다.

6. Active agent와 passive agent

Active agent는 stimulus를 보내고 실제 interface도 관찰한다.

개념 흐름

active agent



Passive agent는 다른 주체가 구동하는 interface를 관찰만 한다.

개념 흐름
 passive agent
 └─ monitor

Active agent에도 monitor가 필요하다. Driver transaction은 보내려던 의도를 나타내지만, monitor transaction은 interface에서 실제로 성립한 동작을 나타낸다.

이미 외부 장치가 구동하는 signal을 UVM driver도 구동하면 multiple-driver contention이 발생하고 값이 X가 될 수 있다.

7. Sequencer, driver, monitor

Sequencer는 sequence 자체를 driver로 보내는 것이 아니다. Sequence가 만든 transaction의 전달을 중재한다.

개념 흐름

Sequence	→ 무엇을 어떤 순서로 보낼지 결정
Sequencer	→ transaction 전달을 중재
Driver	→ transaction을 pin-level signal로 변환
Monitor	→ 실제 pin-level signal을 transaction으로 복원

Monitor는 driver의 원래 transaction을 그대로 받지 않는다. 실제 interface를 독립적으로 관찰해야 driver 오류, handshake 실패, DUT와의 실제 상호작용을 검출할 수 있다.

8. Analysis port와 subscriber

Monitor는 관찰한 transaction을 analysis port로 발행한다.

개념 흐름

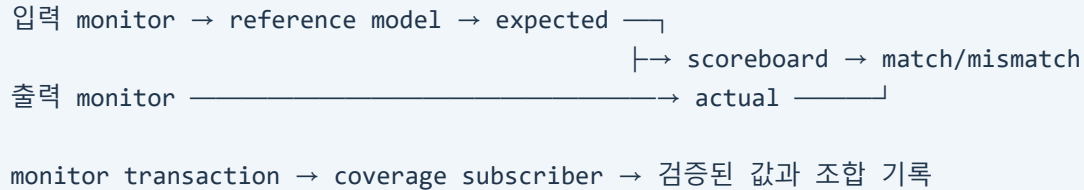
```

monitor — analysis —┬─> scoreboard
                    ┌─> coverage subscriber
                    └─> logger
  
```

Analysis port는 독립적인 component가 아니라 transaction을 1:N으로 방송하는 통신 통로다. 구체적인 연결 문법은 TLM 단계에서 다룬다. Subscriber는 transaction을 구독하여 처리하는 component다. Coverage에만 한정되지 않으며 logging이나 통계 수집에도 사용할 수 있다.

9. Reference model, scoreboard, coverage

개념 흐름



- Reference model: 입력 transaction으로 올바른 expected result를 계산한다.
- Scoreboard: expected와 actual을 비교하여 correctness를 판단한다.
- Coverage subscriber: 무엇을 얼마나 시험했는지 측정한다.

작은 환경에서는 prediction과 comparison을 scoreboard 하나에 넣을 수 있다. 큰 환경에서는 reference model과 scoreboard를 분리하면 각각 교체하고 재사용하기 쉽다.

모든 비교가 pass여도 coverage가 낮으면 검증이 충분하지 않을 수 있다. 반대로 coverage가 높아도 결과 비교가 없다면 DUT가 올바르게 동작했다고 판단할 수 없다.

10. Virtual sequence와 virtual sequencer

여러 agent의 동작을 하나의 복합 시나리오로 조정할 때 사용한다.

개념 흐름

virtual sequence

| 실행 순서 지휘



virtual sequencer

├─ spi_sequencer handle

├─ uart_sequencer handle

└─ reset_sequencer handle

Virtual sequence는 여러 protocol sequence의 실행 순서를 결정한다.

개념 흐름

1. SPI 설정 sequence 실행

2. UART sequence 실행

3. Reset sequence 실행

Virtual sequencer는 각 protocol의 실제 sequencer handle을 보관한다. Sequence들의 집합이 아니며, 일반적으로 자기 driver가 없고 pin을 직접 구동하지 않는다.

개념 흐름

virtual sequence → SPI sequence → SPI sequencer → SPI driver → SPI pin

실제 pin timing은 각 protocol driver가 담당한다.

11. 생성, hierarchy, 수명, 책임

대상	일반적인 생성 주체	UVM hierarchy	일반적인 수명	Transaction 역할	Pin 접근	Correctness 판단
Test	UVM	있음	Simulation 전체	시나리오 선택	안 함	보통 안 함
Environment	Test	있음	Simulation 전체	구조와 연결	안 함	안 함
Agent	Environment	있음	Simulation 전체	Protocol 묶음	안 함	안 함
Sequence	Test/virtual sequence 등	없음	실행 기간	생성·순서 결정	안 함	안 함
Transaction	Sequence/monitor	없음	전달·처리 기간	전달되는 데이터	안 함	안 함
Sequencer	Agent	있음	Simulation 전체	중재·전달	안 함	안 함
Driver	Agent	있음	Simulation 전체	Signal로 변환	구동	안 함
Monitor	Agent	있음	Simulation 전체	Transaction으로 복원	관찰	보통 안 함
Scoreboard	Environment	있음	Simulation 전체	Expected/actual 비교	안 함	함

대상	일반적인 생성 주체	UVM hierarchy	일반적인 수명	Transaction 역할	Pin 접근	Correctness 판단
Coverage subscriber	Environment	있음	Simulation 전체	검증 범위 측정	안 함	안 함
Virtual sequencer	Environment	있음	Simulation 전체	Sequencer handle 제공	안 함	안 함
Virtual sequence	Test 등	없음	실행 기간	여러 sequence 지휘	안 함	안 함

Interface와 DUT는 UVM component hierarchy가 아니라 top의 HDL hierarchy에 존재하며 simulation 동안 유지된다.

자주 틀렸던 부분

1. Sequence는 UVM class지만 UVM component hierarchy에는 존재하지 않는다.
2. Sequencer는 sequence 객체를 driver에 보내는 것이 아니라 transaction 전달을 중재한다.
3. Driver의 transaction은 의도이고 monitor의 transaction은 interface에서 실제로 관찰된 결과다.
4. Monitor는 일반적으로 correctness를 판정하지 않는다. 비교는 scoreboard가 담당한다.
5. Analysis port는 판단 주체가 아니라 transaction 방송 통로다.
6. Subscriber는 coverage 전용 이름이 아니다. Coverage subscriber는 subscriber의 한 사용 방식이다.
7. Virtual sequence는 실행 순서를 지휘한다.
8. Virtual sequencer는 virtual sequence handle이 아니라 실제 sequencer handle을 보관한다.
9. Virtual sequencer는 일반적으로 자기 driver가 없으며 pin을 직접 구동하지 않는다.

현재 이해도와 다음 단계

- UVM 전체 transaction 흐름: 3/5
- Directed와 constrained-random 구분: 3/5
- Transaction-level verification과 reuse: 3/5
- Test, environment, agent 구조: 3/5
- Active/passive agent 구분: 3/5
- Sequencer, driver, monitor 역할: 3/5
- Reference model, scoreboard, coverage 역할: 2~3/5
- Virtual sequence와 virtual sequencer: 2/5

다음 학습 주제는 6단계 `uvm_object`와 `uvm_component`다.

PDF 보충 - 생성·실행·연결을 따로 설명하기

2026-10-07 보강. 연결: 02.01, 책 147~152쪽; 02.02, 책 154~156쪽; 02.03, 책 157~164쪽; 02.04, 책 165~166쪽.

UVM은 library와 실행 framework다

SystemVerilog class library를 import하고 macro를 include해 기반 class/API를 사용한다. 등록된 test를 `run_test()`가 선택·생성하고 component hierarchy에 phase를 적용한다. 사용자는 phase method를 구현하며 보통 이를 직접 호출하지 않는다. Test 선택과 종료 절차 상세는 8·13단계로 연결한다.

생성, 실행, 연결은 서로 다른 동작이다

대상	생성	실행 계기	연결
Component	build에서 create(name, parent)	UVM scheduler의 phase 호출	connect에서 port 연결
Sequence	필요할 때 create(name)	start(sequencer)	사용할 sequencer 지정
Transaction	sequence/monitor에서 create(name)	사용자 method 호출	handle을 통신 경로로 전달
Interface	top에서 정적 instance	signal/process 동작	DUT port와 virtual interface

전체 화살표를 하나의 호출 체인이나 hierarchy로 오해하지 않는다. Sequence의 실행과 driver-monitor의 phase process는 병행된다. Driver를 교체하는 factory 설정, 설정값을 전달하는 config DB, 통신 port 연결은 각각 다른 작업이다.

Request와 response는 역할 이름이다

Transaction은 입력 stimulus에만 사용되지 않는다. 응답, 관찰 결과, 상태도 의미 있는 object로 모델링할 수 있다. PDF의 reactive 흐름에서 DUT 요청을 받아 response를 만드는 경로도 소개하지만, 일반적인 FIFO 입력 생성 환경과 동일한 책임 배치를 강제하지 않는다.

UVM을 두 축으로 설명할 때의 범위

Component 쪽은 환경 구성·상시 구동/관찰, sequence 쪽은 시나리오 실행이라는 관점으로 구조를 설명할 수 있다. 엄밀한 class 전체 분류는 아니며 transaction, configuration object, RAL model처럼 sequence 이외의 object도 있다. Callback·RAL·TLM-2는 후속 보충 주제로 남긴다.

보충 확인 문제: driver 객체 생성, driver.run_phase 실행, sequencer port 연결은 각각 언제 누가 수행하는가? 새 보충 문제이며 답변·실행은 아직 확인하지 않았다.

SystemVerilog/UVM 6단계 - uvm_object와 uvm_component

Class 상속 (extends)

```

uvm_object
↓
uvm_report_object
↓
uvm_component
↓
uvm_driver #(fifo_item)
↓
fifo_driver
  
```

Component 배치 (parent)

```

uvm_test_top
↓
env
↓
agent
↓
driver
  
```

Object create의 context용 parent는 component hierarchy 자식 등록과 구분한다.

학습 범위: UVM class를 object 계열과 component 계열로 분류하고, 각 계열의 hierarchy, parent, phase, 수명, 역할, constructor 차이를 이해한다.

1. 이번 단계의 핵심

UVM class는 크게 두 부류로 나눠 이해할 수 있다.

개념 흐름

uvm_object 계열

→ transaction과 sequence처럼 전달되거나 실행되는 대상

uvm_component 계열

→ testbench hierarchy를 구성하고 simulation 동안 역할을 수행하는 구성 요소

간단한 판단 기준은 다음과 같다.

- hierarchy 안에 고정된 위치가 필요한가?
- component의 parent가 필요한가?
- UVM phase scheduler가 method를 호출해야 하는가?
- transaction처럼 필요할 때 만들고 처리하는 대상인가?

2. uvm_object 계열

대표적인 object 계열 class는 다음과 같다.

```

개념 흐름
uvm_object
├── uvm_transaction
│   ├── uvm_sequence_item
│   │   ├── bus_item 같은 transaction
│   │   └── uvm_sequence_base
│   │       ├── uvm_sequence
│   │       └── write_sequence 같은 sequence

```

상속 구조는 UVM 버전과 class 종류에 따라 더 세부적일 수 있다. 여기서는 sequence item과 sequence가 모두 **uvm_object** 계열이라는 점이 중요하다.

uvm_sequence_item

한 번의 요청이나 전송을 표현하는 transaction의 기반 class다.

SystemVerilog / UVM

```
class bus_item extends uvm_sequence_item;
    rand bit        write;
    rand bit [7:0]  addr;
    rand bit [7:0]  data;

    function new(string name = "bus_item");
        super.new(name);
    endfunction
endclass
```

`bus_item`은 "무슨 동작을 할지"를 담는다. Protocol pin timing은 보통 driver가 담당한다.

uvm_sequence

Transaction을 만들고 어떤 순서로 보낼지 표현하는 시나리오의 기반 class다.

SystemVerilog / UVM

```
class write_sequence extends uvm_sequence #(bus_item);
    task body();
        // bus_item을 만들어 전송하는 시나리오
    endtask
endclass
```

Sequence는 UVM class이지만 component hierarchy에 들어가지 않는다. 생성만으로 `body()`가 실행되는 것도 아니다. Sequence를 `start(sequencer)`해야 시나리오가 실행된다.

Object 계열 특징

- component hierarchy에 속하지 않는다.
- component parent를 갖지 않는다.
- UVM phase scheduler가 `build_phase()`나 `run_phase()`를 자동 호출하지 않는다.

- 필요한 시점에 만들고 전달·처리할 수 있다.
- 이름이 있어도 component instance path가 생기지는 않는다.

3. uvm_component 계열

대표적인 component 계열 class는 다음과 같다. 아래 목록은 계열을 보여주기 위한 것으로 중간 상속 class를 생략했다.

개념 흐름

```

uvm_component
├── uvm_test
├── uvm_env
├── uvm_agent
├── uvm_sequencer
├── uvm_driver
├── uvm_monitor
├── uvm_scoreboard
└── uvm_subscriber

```

이 class들은 모두 component 계열이지만 서로가 모두 부모·자식 class인 것은 아니다. 각 class가 uvm_component를 바탕으로 서로 다른 역할을 제공한다.

Component hierarchy 예

개념 흐름

```

uvm_test_top
├── env
│   ├── agent
│   │   ├── sequencer
│   │   ├── driver
│   │   └── monitor
└── scoreboard

```

이 hierarchy는 component 생성 때 parent 관계를 전달하여 만든다. 생성된 driver는 agent 객체 아래에 배치된다. Component는 보통 환경 구성 중 생성되어 simulation 동안 유지되고 phase에 참여한다.

4. 상속 관계와 hierarchy 배치 관계

이 둘은 서로 다른 관계다.

Class 상속 관계

SystemVerilog / UVM

```
class bus_driver extends uvm_driver #(bus_item);
```

extends는 class 설계도 사이의 상속을 표현한다.

개념 흐름

bus_driver class → uvm_driver class의 기능을 상속

생성된 component의 부모·자식 관계

SystemVerilog / UVM

```
driver = bus_driver::type_id::create("driver", this);
```

이 코드가 agent 안에 있다면 this는 현재 agent 객체다. 생성된 driver component는 해당 agent의 자식으로 배치된다.

개념 흐름

bus_agent object

└─ bus_driver object

따라서 "driver가 agent 아래에 있다"는 사실만으로 driver class가 agent class를 상속하는 것은 아니다.

5. Constructor 인수 차이

Object constructor는 보통 이름만 받는다.

SystemVerilog / UVM

```
function new(string name = "bus_item");
    super.new(name);
endfunction
```

Component constructor는 이름과 parent를 받는다.

SystemVerilog / UVM

```
function new(string name, uvm_component parent);
    super.new(name, parent);
endfunction
```

- **name**: 생성할 object 또는 component의 이름
- **parent**: hierarchy에서 component를 포함할 상위 component 객체

Transaction에는 component hierarchy 위치가 없으므로 parent가 필요 없다. Driver, monitor, agent 등은 hierarchy에 놓여야 하므로 parent가 필요하다.

중요한 표현 차이:

개념 흐름

parent는 bus_env class 자체가 아니다.
parent는 실제로 생성된 bus_env 객체(this)다.

일반적인 run_test() 경로에서 test는 parent 인수로 null을 전달하고 UVM root 아래에 배치된다. 그 아래 component는 상위 component 객체를 parent로 받는다.

6. 이름과 instance path

Component 생성에서 class 이름과 instance 이름을 구분한다.

SystemVerilog / UVM

```
bus_driver::type_id::create("driver", this);
```

개념 흐름

bus_driver → class type

"driver" → 생성되는 component instance 이름

this → parent component 객체

Environment의 경로가 `uvm_test_top.env`라면 위 driver의 경로는 다음과 같다.

개념 흐름

`uvm_test_top.env.agent.driver`

같은 parent 아래에 같은 instance 이름의 component를 중복 생성하면 경로가 충돌한다. 두 agent는 다른 이름을 사용해야 한다.

SystemVerilog / UVM

```
agent0 = bus_agent::type_id::create("agent0", this);
```

```
agent1 = bus_agent::type_id::create("agent1", this);
```

Transaction object는 component hierarchy에 등록되지 않으므로 이름이 같아도 이 component 경로 충돌은 발생하지 않는다.

7. Phase와 실행 방식

UVM은 component hierarchy에 있는 component의 phase method를 정해진 순서로 호출한다.

개념 흐름

`build_phase()` → component 구성과 설정
`connect_phase()` → 통신 연결
`run_phase()` → 시간에 따른 동작

Driver와 monitor의 `run_phase()`는 UVM이 실행한다. Sequence는 component가 아니므로 `run_phase()`가 자동 호출되지 않는다.

개념 흐름**driver 생성**

→ component hierarchy에 등록
→ UVM이 `driver.run_phase()` 호출

sequence 생성

→ 아직 시나리오는 실행되지 않음
→ `sequence.start(sequencer)`
→ `sequence.body()` 실행

Sequence의 `body()`는 component phase와 별개의 sequence 실행 method다.

8. 역할과 계열 한눈에 보기

Class 또는 대상	계열	Hierarchy	핵심 역할
bus_item	uvm_object	없음	transaction 한 건의 데이터
write_sequence	uvm_object	없음	transaction 생성과 시나리오 순서
bus_sequencer	uvm_component	있음	sequence 요청을 중재하고 item 전달
bus_driver	uvm_component	있음	transaction을 pin 동작으로 변환
bus_monitor	uvm_component	있음	실제 pin을 관찰해 transaction 복원
bus_agent	uvm_component	있음	한 protocol의 구성 요소 묶음
bus_env	uvm_component	있음	agent, scoreboard 등의 검증환경 구성
bus_test	uvm_component	있음	사용할 환경과 시나리오 선택
bus_scoreboard	uvm_component	있음	expected와 actual 비교
Coverage subscriber	uvm_component	있음	transaction을 받아 coverage 기록

9. 자주 혼동한 부분

- 1. Sequence는 UVM class이지만 component hierarchy에는 없다.
- 2. Sequencer는 sequence들의 집합이 아니라 item 전달을 중재하는 component다.
- 3. Sequence는 body()가 실행되고, driver는 UVM phase의 run_phase()에 참여한다.

4. Object constructor에는 component parent가 없고 component constructor에는 parent가 있다.
5. `extends`는 class 상속이고, `create(..., this)`는 생성된 component의 hierarchy 배치다.
6. `create()`는 이름이 같은 기존 객체를 찾아 반환하는 기능이 아니다. 중복 component 이름은 hierarchy 충돌을 일으킨다.
7. Scoreboard는 driver가 보내려 한 transaction이 아니라 monitor가 실제 interface에서 관찰한 transaction을 받아 비교하는 것이 일반적이다.
8. Coverage subscriber는 coverage를 기록하며 correctness의 최종 판정은 보통 scoreboard가 담당한다.

10. 이해도와 다음 단계

현재 이해도: 3/5

- 예제를 보고 object와 component를 분류할 수 있다.
- Constructor의 name/parent 차이를 설명할 수 있다.
- Class 상속과 component hierarchy 배치 관계를 구분할 수 있다.
- Sequence의 `body()`와 component의 phase 실행 차이를 설명할 수 있다.
- 다음에는 class 계열과 생성 방식을 더 정확히 연결하면 된다.

다음 학습 주제는 7단계 **Factory**와 **utility macro**다.

PDF 보충 - 상속과 hierarchy의 정확한 경계

2026-10-07 보강. 연결: 02.01, 책 147~152쪽; 02.04, 책 165~166쪽; 02.06 Logical Hierarchy, 책 190~191쪽.

Component도 상속상 `uvm_object`의 후손이다

개념 흐름

`uvm_object`

```
├─ uvm_transaction → uvm_sequence_item → item/sequence 계열
└─ uvm_report_object → uvm_component → test/env/agent/driver 등
```

여기서 object 계열/component 계열이라는 실무 분류는 hierarchy와 phase 책임을 구분하는 표현이다. 상속상 완전히 독립된 두 뿌리가 아니다. uvm_component도 object 기능을 상속하지만 component 전체를 transaction처럼 clone해 hierarchy를 복제하는 용도로 사용하지 않는다.

Object create()의 parent 인수와 hierarchy

SystemVerilog / UVM

```
item = fifo_item::type_id::create("item", this);
```

Object registry의 create()는 선택적으로 component를 받아 factory 검색 context에 사용할 수 있다. 이 인수가 object constructor의 parent가 되거나 object를 component hierarchy의 자식으로 등록하는 것은 아니다. Object의 new(name)과 component의 new(name,parent)를 구분한다.

UVM root와 test parent

일반적인 run_test() 경로는 root 아래에 uvm_test_top을 만든다. Constructor에 parent=null을 전달하면 UVM component 기반 class가 root 아래의 최상위 component로 관리한다. 사용자 환경의 env, agent, driver는 각 생성된 부모 object를 전달한다. Root를 HDL top module과 혼동하지 않는다.

이름을 얻는 API

Component의 get_name()은 leaf 이름, get_full_name()은 component hierarchy의 전체 경로, get_type_name()은 class type 이름을 나타낸다. Factory/config/debug에 쓰는 경로와 class type 문자열은 서로 다른 기준이다. Object의 이름이나 sequence의 실행 context를 component의 구조적 parent-child 관계로 해석하지 않는다.

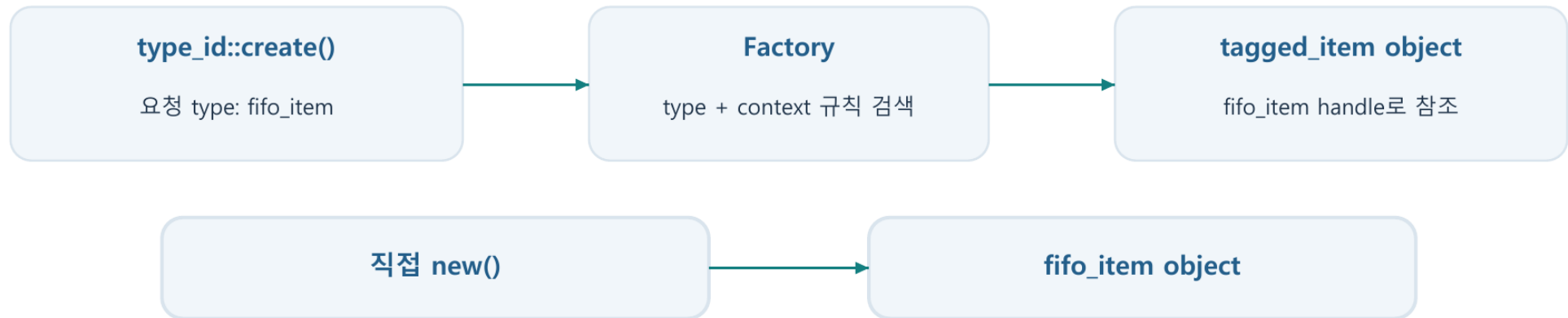
Constructor를 누가 언제 부르는가

Factory는 선택된 type의 constructor를 호출한다. Component의 constructor는 parent 관계를 만들고, build_phase()는 UVM scheduler가 나중에 호출해 하위 구조와 설정을 구성한다. Constructor, build_phase(), run_phase()를 같은 실행 단계로 생각하지 않는다.

보충 확인 문제: fifo_item::type_id::create("item", this)에서 this를 넘겼다는 이유로 item이 component hierarchy에 들어가는가? 새 보충 문제이며 답변·실행은 아직 확인하지 않았다.

SystemVerilog/UVM 7단계 - Factory와 utility macro

fifo_item → tagged_item override를 생성 요청 전에 설정



이미 생성된 object는 바뀌지 않는다. Utils 등록과 field 자동화는 각각의 기능이다.

정리일: 2026-10-07. 대화에서 학습한 이론과 코드 예측을 정리했다. UVM simulator 실행은 미실행이며, PDF에서 새로 보충한 내용은 별도로 표시한다. 연결 자료: 02.06 Factory, 책 197~205쪽; 02.04 Transaction methods/override/parameterized transaction, 책 168~174쪽. PDF 파일 페이지 = 책 쪽수 + 11.

1. Factory의 필요성과 생성 흐름

Factory는 생성 요청을 받아 실제로 생성할 type을 선택한다. 공통 환경 안에 생성 코드가 여러 곳 있어도 test에서 설정을 바꿔 특수한 transaction이나 driver를 사용할 수 있다. Factory는 기존 object를 교체하는 기능이 아니며 다음 생성 요청에 적용한다.

개념 흐름

class 등록 → override 설정 → type_id::create() 요청

→ 경로/type 규칙 확인 → 선택한 type의 constructor → handle 반환

직접 new()를 호출하면 해당 handle type의 constructor를 사용하고 factory override 검색은 하지 않는다. Factory create()도 실제 객체 생성에는 결국 constructor를 사용한다.

2. 등록, type_id, constructor

SystemVerilog / UVM

```
class fifo_item extends uvm_sequence_item;
  `uvm_object_utils(fifo_item)
  rand bit [7:0] data;
  function new(string name = "fifo_item");
    super.new(name);
  endfunction
endclass
```

Utils macro는 전처리 단계에서 factory 등록과 type 관련 method/boilerplate를 마련한다. 매크로를 쓴 자리에서 transaction instance를 만드는 것이 아니다. type_id는 등록용 registry specialization에 붙인 typedef 이름이며, 이 registry가 lightweight type proxy를 factory에 제공한다. 사용자가 만드는 fifo_item object와 proxy singleton은 다른 대상이다.

Object에는 uvm_object_utils, component에는 uvm_component_utils를 사용한다. Object constructor는 name, component constructor는 name과 parent를 받는다. UVM 1.2 object factory용 constructor에는 name 인수를 준비한다.

SystemVerilog / UVM

```
// agent의 build_phase 안: this는 생성된 agent 객체
drv = fifo_driver::type_id::create("driver", this);
```

위 parent는 component hierarchy를 정한다. Component 등록 매크로는 적합한 component registry를 마련하며 필수 constructor를 대신 작성하지 않는다.

3. Type override와 적용 시점

tagged_item이 fifo_item을 상속하고 두 class가 등록돼 있다고 가정한다.

SystemVerilog / UVM

```
fifo_item a, b, c;
a = fifo_item::type_id::create("a");
fifo_item::type_id::set_type_override(tagged_item::get_type());
b = fifo_item::type_id::create("b");
c = new("c");
```

Handle	실제 object type	선택 이유
a	fifo_item	override 설정 전에 생성
b	tagged_item	설정 뒤 factory로 생성
c	fifo_item	직접 new()로 생성

세 handle의 선언 type은 fifo_item이다. b만 실제 tagged_item object를 가리키는 것은 부모 handle이 자식 object를 참조할 수 있기 때문이다. Override 설정은 a를 소급해서 바꾸지 않는다. Method overriding은 호출할 구현을 선택하고, factory override는 생성할 type을 선택한다.

4. Instance override와 경로

SystemVerilog / UVM

```
fifo_driver::type_id::set_type_override(traced_driver::get_type());
fifo_driver::type_id::set_inst_override(
    fault_driver::get_type(), "uvm_test_top.env.agent1.driver"
);
```

이 설정 후 agent0.driver와 agent1.driver가 fifo_driver로 생성 요청을 하면 agent0에는 traced_driver, agent1에는 fault_driver가 생성된다. 일치하는 instance override가 type override보다 우선한다. Instance override를 제거해도 type override는 남는다. 대체 type은 요청 type과 대입 호환되도록 설계한다.

PDF 보충: context와 상대 경로

Registry `set_inst_override()`에서 `parent`를 생략하면 경로를 절대 경로로 해석한다. Parent를 넘기면 그 component의 전체 이름을 앞에 붙인다. Test의 component method인 `set_inst_override_by_type()`는 test 기준 상대 경로를 받는다. 서로 다른 API의 인수 순서와 경로 기준을 혼용하지 않는다.

Object는 구조적 component `parent`가 없지만 factory 생성 context를 가질 수 있다. Object `create(name,parent,contxt)`의 `context/name`과 `override` 경로가 맞아야 한다. Wildcard는 여러 요청에 적용될 수 있으므로 실제 경로로 먼저 확인한다. 여러 instance 규칙이 겹치면 등록 순서에도 주의한다.

5. Utils와 field 자동화

매크로	핵심 역할
<code>uvm_object_utils(T)</code>	object의 factory/type 관련 기능
<code>uvm_component_utils(T)</code>	component의 factory/type 관련 기능
<code>uvm_object_utils_begin/end</code>	object 등록 + field macro 구간
<code>uvm_component_utils_begin/end</code>	component 등록 + field macro 구간
<code>uvm_field_*</code>	field별 copy/compare/print/pack 등 자동화

SystemVerilog / UVM

```
`uvm_object_utils_begin(fifo_item)
  `uvm_field_int(data, UVM_DEFAULT)
  `uvm_field_int(debug_id, UVM_DEFAULT | UVM_NOCOMPARE)
`uvm_object_utils_end
```

`uvm_field_int`는 `int` keyword field에만 쓰는 것이 아니라 `bit/logic packed integral field`에도 사용한다. `UVM_DEFAULT`는 기본 작업들을 켜다. `NOCOMPARE`를 지정한 `debug_id`는 비교에서는 제외되지만 `copy/print` 등은 별도 flag가 없으면 참여한다. Field macro 구간을 단순 `utils`로 바꾸면

factory 등록은 유지되고 사용자 field 자동화는 빠진다.

PDF 보충: field 종류

uvm_field_object/string/enum, uvm_field_array_*, uvm_field_queue_* 등 type에 맞는 macro를 고른다. UVM_HEX/DEC 등은 표시 radix 정책이다. UVM_NOCOPY/NOPRINT/NOPACK은 작업별 참여 여부를 조정한다. Begin/end와 component/object 계열을 맞춘다.

6. copy()와 do_copy()

사용자는 목적지.copy(원본)을 호출한다. UVM은 자동화와 virtual hook을 통해 field를 처리하며, 수동 구현은 do_copy()에 둔다. 목적지는 호출 전에 유효한 object여야 한다.

SystemVerilog / UVM

```
function void do_copy(uvm_object rhs);
    fifo_item src;
    super.do_copy(rhs);
    if (!$cast(src, rhs))
        `uvm_fatal("COPY", "rhs is not a fifo_item")
    this.data = src.data;
endfunction
```

rhs의 선언 type은 uvm_object이므로 fifo_item 고유 field에 접근하려면 \$cast가 필요하다. \$cast는 type 호환성을 검사하고 handle을 얻으며 객체를 복제하지 않는다. a.copy(b) 후 b.data를 변경해도 a.data는 복사 당시 값으로 유지된다. a=b는 handle assignment이며 별개의 연산이다.

7. compare()와 do_compare()

compare()는 같으면 1, 다르면 0을 반환한다. 어떤 field를 비교할지는 field macro와 do_compare() 구현으로 정한다. 아래는 대화에서 사용한 간단한 값 비교 예다.

SystemVerilog / UVM

```
function bit do_compare(uvm_object rhs, uvm_comparer comparer);
    fifo_item other;
    if (!$cast(other, rhs)) return 0;
    return super.do_compare(rhs, comparer)
        && (this.data == other.data);
endfunction
```

super.do_compare()는 같은 두 object의 부모 class 구현을 호출한다. 부모가 addr, 자식이 data를 비교하도록 구현돼 있다면 두 조건이 모두 참이어야 한다. 비교 함수가 같은지 확인하는 것이 아니다. Debug ID가 field macro나 수동 코드에 포함돼 있으면 값이 달라 실패할 수 있고, 제외했다면 비교 결과에 영향을 주지 않는다.

PDF 보충: comparer를 쓰는 구현

실제 환경에서는 comparer.compare_field_int() 등으로 mismatch 정보와 정책을 함께 반영할 수 있다. 모든 필요한 비교를 수행하려면 short-circuit 때문에 뒤의 비교가 생략되지 않게 작성한다.

SystemVerilog / UVM

```
function bit do_compare(uvm_object rhs, uvm_comparer comparer);
    fifo_item other;
    bit ok;
    if (!$cast(other, rhs)) return 0;
    ok = super.do_compare(rhs, comparer);
    ok &= comparer.compare_field_int("data", data, other.data, $bits(data));
    return ok;
endfunction
```

이 확장 구현은 자료 보충이며 직접 작성·실행을 확인한 것은 아니다. 같은 field를 field macro와 do_*에서 중복 처리하지 않도록 정책을 정한다.

8. do_print(), sprint(), convert2string()

SystemVerilog / UVM

```
function void do_print(uvm_printer printer);
    super.do_print(printer);
    printer.print_field_int("data", data, $bits(data), UVM_HEX);
endfunction

function string convert2string();
    return $sformatf("data=0x%02h", data);
endfunction
```

print()는 printer를 사용해 출력하고, sprint()는 같은 형식의 문자열을 반환한다. convert2string()은 사용자 정의 한 줄 문자열을 반환하며 직접 화면에 출력하지 않는다. Utils/field macro는 convert2string()의 사용자 문자열을 자동 작성하지 않는다. 반환값을 uvm_info나 \$display에 전달하면 로그로 출력된다.

9. clone()과 중첩 object 복사

clone()의 기본 구현은 새로운 object를 생성하고 copy()로 내용을 복사한다. 반환 type은 uvm_object이므로 구체 handle에 담을 때 \$cast한다. 바깥 object가 새로 생겨도 내부 handle의 공유 여부는 복사 구현에 달려 있다.

SystemVerilog / UVM

```
fifo_item a, b;
a = fifo_item::type_id::create("a");
a.data = 20;
if (!$cast(b, a.clone()))
    `uvm_fatal("CLONE", "Clone type mismatch")
b.data = 30; // data가 올바르게 복사됐다면 a.data는 20 유지
```

기본 clone()에서 호출하는 virtual object create()와 factory registry의 type_id::create()는 서로 다른 method다. Clone을 새로운 factory override 검색 요청으로 일반화하지 않는다.

SystemVerilog / UVM

```
this.hdr = src.hdr; // 내부 object 공유
// this.hdr = new() 뒤 위 대입을 해도 공유로 돌아감
```

SystemVerilog / UVM

```
if (src.hdr == null) this.hdr = null;
else begin
    this.hdr = header::type_id::create("hdr");
    this.hdr.copy(src.hdr);
end
```

두 번째 코드는 header의 복사 구현도 올바르다는 가정에서 독립적인 내부 내용을 만든다. Destination에 이미 다른 handle과 공유 중인 내부 object가 있는 경우도 고려해 정책을 선택한다.

정책	object field 복사 의미	내부 id 수정의 영향
UVM_REFERENCE	handle을 복사해 같은 object 참조	원본에서도 보임
UVM_DEEP	내부 object의 내용까지 복사	올바른 구현이면 독립
UVM_SHALLOW	중첩 내용을 얇게 처리하는 정책	deep과 같다고 가정하지 않음

REFERENCE/DEEP은 대화에서 혼동한 뒤 재확인했다. SHALLOW의 구체 동작과 comparer/packer별 정책은 사용 UVM 구현에서 추가 확인할 보충 항목이다.

10. Field 자동화의 선택과 한계

Macro는 많은 작업을 지원하는 코드를 생성하므로 사용 범위가 넓을수록 불필요한 처리와 정책 분기가 들어갈 수 있다. 실제 성능 영향은 field 수, 내부 object 구조, 호출 빈도, simulator에 따라 측정해야 한다. 직접 구현보다 항상 느리다는 단정이나 속도 수치는 사용하지 않는다.

전처리 확장 코드 안의 오류는 원래 field 선언만 보고 찾기 어려울 수 있다. Nested object의 null/공유 정책과 비교 제외 조건이 복잡하면 do_*가 의도를 드러내기 좋다. 작은 단순 transaction에는 field macro가 편리하다. 실수로 field를 누락하면 복사·비교가 성공처럼 보일 수 있으므로 포함할 field 목록을 검증 계획과 맞춘다.

11. Factory debug와 PDF 설명 보정

확인 순서: 등록 여부 → 생성이 create인지 → override 설정 시점 → 실제 검색 경로 → 적용 규칙. Factory의 print()는 등록과 override 상태, debug_create_by_type()는 생성 없이 선택 규칙을 추적한다.

```
SystemVerilog / UVM
uvm_factory f;
f = uvm_factory::get();
f.print();
f.debug_create_by_type(fifo_driver::get_type(),
    "uvm_test_top.env.agent1", "driver");
```

실제 생성 type은 get_type_name(), component 위치는 get_full_name()으로 함께 확인한다. Debug API 실습 로그는 아직 없다.

PDF 책 199쪽의 type_id는 typedef를 통해 registry type을 가리킨다. 이를 transaction singleton instance로 해석하지 않는다. 책 200쪽의 component create 예에는 실제 코드에서 parent가 필요하다. 책 169쪽의 do_pack/do_unpack signature는 UVM 1.2의 [virtual function void do_pack\(uvm_packer packer\)](#) 및 do_unpack과 구분한다. 책의 도식에 등록·생성·override가 나열돼 있어도 override가 해당 생성 요청보다 먼저 설정돼야 한다.

12. PDF 보충 - Parameterized transaction

SystemVerilog / UVM

```
class bus_item #(int DW = 8) extends uvm_sequence_item;
  `uvm_object_param_utils(bus_item#(DW))
  rand bit [DW-1:0] data;
  function new(string name = "bus_item");
    super.new(name);
  endfunction
endclass
typedef bus_item#(16) bus_item16_t;
```

Parameterized class에는 object/component_param_utils 계열을 사용한다. DW=8과 DW=16 specialization은 다른 type이다. Name 기반 factory 등록·검색은 일반 class와 같다고 가정하지 않고 type 기반 API를 사용한다. 이 내용은 PDF에서 보충했으며 문제·실행으로 확인하지 않았다.

13. 학습 확인과 다음 시작점

현재 이해도: 기본 생성·override와 scalar field 처리 2~3/5. 직접 전체 class 작성·UVM simulator 디버깅은 미평가다.

- 확인됨: create/new의 차이, 이미 생성된 객체 불변, instance 우선순위, NOCOMPARE, copy와 handle assignment, clone의 바깥 객체 독립, 문자열 반환.
- 재확인 필요: REFERENCE/DEEP과 내부 공유, super.do_compare의 부모 field 재사용, 실제 object override 경로.
- 자료 보충: parameterized registration, comparer 정책, proxy 구조, debug API, do_pack/unpack 보정.
- 미실행: 이 노트의 모든 UVM 예제. 실제 simulator pass 로그 없음.

반드시 기억할 핵심 3개: factory 설정은 생성 전에 / utils 등록과 field 자동화는 별개 / 바깥 clone과 내부 deep copy는 별개.

복습 문제 3개:

1. Override 뒤 create와 직접 new는 어떤 type을 생성하는가?
2. super.do_compare가 0이고 data가 같으면 전체 결과는 무엇인가?
3. UVM_REFERENCE로 복사한 내부 object를 수정하면 원본에도 영향이 있는가?

답변을 가린 상태로 먼저 설명한 뒤 본문과 비교한다. 다음 관련 단계는 SystemVerilog UVM 8단계 - Phase와 objection이다.

참고 자료

- 회사 PDF: 02.04, 02.06의 위 쪽수. 본문을 재구성하고 필요한 API를 보정했다.
- UVM 1.2 Factory
- UVM 1.2 Registry
- UVM Utility/Field Macros
- UVM Object
- UVM Comparer

SystemVerilog/UVM 8단계 - Phase와 objection



forever task 자체는 phase를 유지하지 않는다. Drain time은 완료 조건 확인을 대신하지 않는다.

정리일: 2026-10-07. 대화에서 phase·종료 흐름을 설명하고 짧은 코드 해석으로 확인했다. Simulator 실행은 미실행이다. 고급 phasing과 sequence 자동 objection은 PDF 보충이며 직접 실습하지 않았다. 연결: 02.01 UVM Phasing/Hello World, 책 150~153쪽; 02.02 실행, 책 154쪽; 02.09 Phasing, 책 230~237쪽. PDF 파일 페이지 = 책 쪽수 + 11.

1. Phase가 해결하는 문제

검증환경을 만들기 전에 연결하거나 연결이 끝나기 전에 pin 구동을 시작하면 null handle과 연결 누락 등이 생긴다. UVM scheduler는 component hierarchy를 따라 생성·연결·실행·검사·보고를 정해진 phase로 진행한다. 사용자는 해당 virtual phase method를 구현하며 일반적으로 직접 호출하지 않는다.

2. Common phase 순서와 책임

Phase	형식	hierarchy 방향	주요 책임
build	function	부모 → 자식	하위 component 생성, 설정 읽기
connect	function	자식 → 부모	TLM port/export/imp 연결
end_of_elaboration	function	자식 → 부모	구조·설정·연결 최종 확인
start_of_simulation	function	자식 → 부모	실행 전 topology·설정 출력
run	task	component마다 병렬	stimulus, driver 구동, monitor 관찰
extract	function	자식 → 부모	실행 결과·통계 추출
check	function	자식 → 부모	미처리 비교 등 최종 검사
report	function	자식 → 부모	결과 보고
final	function	부모 → 자식	파일 닫기 등 마지막 정리

모든 component의 build가 끝나고 connect로 넘어간다. 같은 phase 안에서의 hierarchy 방문 방향과 phase 사이의 순서는 다른 기준이다. Test → env → agent → driver 구조라면 build는 이 순서로, connect는 반대로 방문한다. Sibling 간 순서를 testbench 동작 조건으로 의존하지 않는다.

PDF 책 231쪽의 build 이외 모두 bottom-up이라는 표현에는 final의 예외를 보충했다. Final도 top-down이다. Build에서 test가 먼저 factory override/config를 설정하고 자식이 나중에 create/get하는 흐름을 만들 수 있다. 부모 build에서 실제 생성 전에 override를 설정하는 순서까지 확인한다.

3. Function/task는 순차/병렬 구분이 아니다

Function은 simulation time을 소비할 수 없고, task는 delay나 clock event를 기다릴 수 있다. Task도 일반 호출이면 호출한 process에서 순차 실행된다.

SystemVerilog / UVM

```
drive(); // 호출 후 10ns에 반환
monitor(); // 그 뒤 호출, 20ns 뒤 반환
// 전체 30ns
```

SystemVerilog / UVM

```
fork
  drive(); // 0~10ns
  monitor(); // 0~20ns
join      // 전체 20ns
```

Run phase가 병렬인 이유는 UVM scheduler가 각 component의 run_phase를 별도 process로 실행하기 때문이다. Task라는 keyword만으로 자동 병렬화하지 않는다. Logical concurrency와 CPU core별 실행 방식은 구분한다.

Driver는 @(posedge vif.clk)처럼 실제 interface clock이나 @(vif.driv_cb)의 clocking event를 기다릴 수 있다. Pin timing 정책은 4단계와 맞춘다.

4. Objection이 필요한 이유

Driver/monitor는 보통 forever loop다. 모든 run_phase의 반환을 기다리는 방식으로는 정상 종료할 수 없다. Objection은 phase에 아직 필요한 일이 있음을 알리는 수명 관리 요청이다. Task가 실행 중이거나 #100ns를 기다린다는 사실만으로 run phase가 유지되지는 않는다.

SystemVerilog / UVM

```
task run_phase(uvm_phase phase);
    phase.raise_objection(this);
    #100ns;
    `uvm_info("TEST", "Done", UVM_LOW)
    phase.drop_objection(this);
endtask
```

Raise와 drop 사이에 phase를 유지하도록 요청한다. 다른 objection과 연계된 종료 조건도 끝나면 종료할 수 있고, 해당 phase의 남은 process는 정리된다. Drop 호출이 즉시 simulator \$finish와 같은 동작은 아니다. 후속 cleanup phase가 진행된다. Objection은 task phase에서 사용한다.

5. 종료 오류 세 가지

오류	원인	결과
Raise 없음	실행 중 task만 믿음	필요한 작업 전 조기 종료 가능
Drop 누락	완료 요청을 내려놓지 않음	phase 유지, timeout까지 대기 가능
너무 이른 drop	요청 전송 완료를 전체 완료로 오해	마지막 출력/비교 유실 가능

Objection은 count로 관리한다. 같은 source가 여러 번 raise했다면 올린 수만큼 drop해야 한다. Raise한 source와 drop source를 맞춘다. 진단용 UVM timeout은 오류를 발견하는 장치이며 정상 완료 조건을 대신하지 않는다.

6. 완료 조건과 wait/if

Stimulus 종료와 DUT 출력 관찰·scoreboard 비교 완료는 다를 수 있다. 더 들어올 요청이 없고 남은 처리가 끝났다는 조건을 함께 사용한다.

SystemVerilog / UVM

```
phase.raise_objection(this);
send_requests(); // 예제용 task: 모든 요청 전송 후 반환한다고 가정
wait (stimulus_done && scoreboard.pending_count == 0);
phase.drop_objection(this);
```

stimulus_done과 pending_count는 개념 예제용 상태다. 실제 환경에서는 monitor의 inflight 관찰, 예상·실제 대기열, latency 등까지 정의한다. Stimulus가 앞으로 더 올 수 있는데 queue가 순간 비었다고 완료로 판단하지 않는다.

if는 그 시점에 한 번 확인한다. 조건이 거짓이라 drop을 건너뛰면 나중에 참이 돼도 자동 재실행되지 않는다. wait는 조건이 참이 될 때까지 대기하며 이미 참이면 즉시 통과한다. Function phase에는 시간 대기 wait를 넣지 않는다.

7. Test 관리와 component별 관리

Test에서 전체 시나리오 시작 전 raise하고 최종 검증 완료 후 drop하는 방식은 책임을 모아 이해하기 쉽다. 규칙상 각 component가 같은 phase에 objection을 올리고 자신의 완료 후 drop할 수도 있다. 모든 source의 objection이 해소돼야 한다.

개념 흐름

```
driver drop 100ns / monitor drop 130ns / scoreboard drop 135ns
→ 다른 지연·objection이 없다면 135ns 이후 종료 가능
```

분산 관리에는 component별 완료 통지와 기준이 필요하다. Driver의 더 받을 요청 없음, monitor의 마지막 관찰 완료, scoreboard의 미래 입력 없음과 비교 완료를 연결해야 한다. Monitor가 phase 종료 후 drop하려고 기다리면 자신의 objection이 종료를 막는 순환 대기가 된다. 단순한 영구 driver/monitor는 objection 없이 동작하고 test가 종료를 관리하는 형태가 흔하다.

8. Drain time과 고정 \$finish

Drain time은 해당 object와 그 하위의 objection count가 모두 내려간 뒤 all_dropped/상위 전파 전에 두는 추가 대기 시간이다. 결과 검사가 완료됐다는 판정이 아니라 여유 시간이다.

SystemVerilog / UVM

```

uvm_objection objection;
objection = phase.get_objection();
objection.set_drain_time(this, 20ns);

```

대화의 `phase.phase_done.set_drain_time(this, 20ns)`와 같은 목적이다. 공개 API `get_objection()`으로 `objection handle`을 얻는 형태도 확인한다. Drain 중 해당 계층에 새 `objection`이 올라가면 진행 중인 drain 종료 절차가 취소되고 이후 다시 평가한다. 계층별 drain time을 중복 지정하면 종료 여유가 단순 한 번의 delay와 달라질 수 있다.

마지막 `drop=100ns`, `drain=20ns`, 새로운 `objection`·추가 `callback` 지연 없음인 단순 예는 120ns 이후 종료할 수 있다. 마지막 출력이 130ns에 오면 보지 못한다. 완료 조건을 확인한 뒤 drop하고 필요한 보조 여유만 둔다.

고정 `#delay` 후 `$finish`는 실제 남은 일을 확인하지 않고 simulation을 강제로 끝낸다. `Objection`은 작업 완료 조건으로 시간을 결정하고 `cleanup phase`를 거쳐 종료한다. 본문의 고정 `delay`는 개념 확인 예이며 실제 검증 완료 기준으로 사용하지 않는다.

9. PDF 보충 - Hello World와 실행 계기

PDF 02.01은 등록된 test의 `run_phase`에서 report를 출력하는 예를 제시한다. Module/program의 initial에서 `run_test()`를 호출하면 UVM이 test를 생성하고 phase를 진행한다. Object 생성만으로 `sequence.body`가 자동 실행되지는 않는다. Sequence 실행은 9단계 `start(sequencer)`에서 다룬다.

UVM report의 ID/severity/verbosity와 `+UVM_TESTNAME`, simulator별 library 옵션은 13단계에서 확대한다. PDF의 VCS 명령을 모든 simulator의 동일 실행 명령으로 취급하지 않는다.

10. PDF 보충 - Runtime phase와 domain

Common `run_phase`와 병행하는 기본 runtime schedule에는 다음 task phase들이 순서대로 있다.

개념 흐름

```

pre_reset → reset → post_reset
→ pre_configure → configure → post_configure
→ pre_main → main → post_main
→ pre_shutdown → shutdown → post_shutdown

```

Runtime schedule 내부의 단계가 모두 한 번에 실행된다는 뜻이 아니다. 같은 phase의 component 동작은 병렬이고 schedule의 단계들은 순서를 따른다. Run과 runtime schedule은 병행하며 extract로 넘어가기 전 관련 종료 조건이 충족돼야 한다. 기본 실습은 run_phase를 중심으로 작성한다.

Domain은 phase schedule을 묶어 서로 다른 block/agent의 동기화 범위를 관리한다. 별도 reset/config 진행이 필요한 환경에서 사용자 domain을 둘 수 있다. Phase jump는 schedule을 앞뒤로 이동할 수 있지만 이전 transaction, queue, monitor, reset state를 정리하도록 환경이 지원해야 한다. Domain/jump는 목적과 위치만 정리한 자료 보충으로, 구현 능력은 확인하지 않았다.

11. PDF 보충 - Sequence의 자동 objection

UVM 1.2에서 set_automatic_phase_objection(1)은 sequence에 유효한 starting phase가 있을 때 실행 주변의 raise/drop을 자동 관리한다. 수동 start를 했다는 이유만으로 starting phase가 항상 자동 지정되는 것은 아니다. 필요하다면 set_starting_phase(phase)로 설정한다.

Forever sequence에 자동 objection을 켜면 정상 drop에 도달하지 못할 수 있다. 자동 관리도 sequence 종료 후 남은 DUT 응답을 자동으로 판정하지 않는다. UVM 1.1의 starting_phase 사용과 1.2의 get/set API를 구분하고, IEEE 1800.2 구현의 호환 옵션을 확인한다. 상세 예제는 9단계에서 실행 흐름과 함께 확인한다.

12. PDF 보충 - Objection debug

개념 흐름

```
+UVM_OBJECTION_TRACE
```

위 plusarg로 raise/drop의 source와 전파를 추적할 수 있다. display_objections()로 남은 source/count를 확인한다. Phase callback인 phase_started(), phase_ready_to_end(), phase_ended()와 objection callback은 종료 흐름을 관찰·확장하는 위치다. 이들 API의 직접 구현은 후속 디버깅 항목이다.

13. 학습 확인과 다음 시작점

현재 이해도: phase 기본 책임과 종료 조건은 2~3/5. 직접 환경 작성·실행은 미평가다.

- 확인됨: task phase의 clock 대기, 별도 process의 병렬 실행, drop 누락, 너무 이른 drop, if/wait의 차이, 분산 objection 관리 가능성, 미래 요청이 있는 빈 queue의 한계.
- 재확인 필요: task가 자동 병렬이라는 혼동, objection 없는 delay의 조기 종료, build top-down과 override 설정 시점의 연결.
- 자료 보충: runtime/domain/jump 개요, starting phase와 자동 objection, callback/trace, final 방향 보정.
- 미실행: 모든 UVM 예제. 이론·코드 예측으로 확인했으며 simulator pass 로그 없음.

반드시 기억할 핵심 3개: function/task는 시간 소비 가능성 / 병렬 실행은 process 구조 / objection은 실제 전체 완료 조건과 연결.

복습 문제 3개:

1. 10ns와 20ns task의 순차 호출과 fork...join 실행 시간은 어떻게 다른가?
2. pending_count가 0이어도 미래 요청이 있다면 drop해도 되는가?
3. Monitor가 phase 종료를 기다린 뒤 drop하려는 설계가 왜 멈추는가?

다음 시작점은 9단계 sequence의 생성과 실행이다. 9단계는 도입 설명만 했고 확인 문제의 답변 및 handshake/FIFO 실습은 아직 진행하지 않았다.

참고 자료

- 회사 PDF: 02.01, 02.02, 02.09의 위 쪽수.
- UVM Common Phases
- UVM Runtime Phases
- UVM Objection
- UVM Phase
- UVM Sequence Base